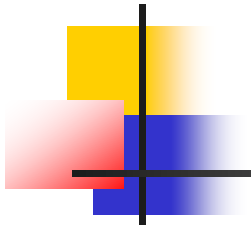
言語・オートマトン
有限状態オートマトン(2)

山本章博
国際高等教育院
2026年度版
連絡先: yamamoto.akihiro.5m@kyoto-u.ac.jp



正則表現

オートマトンから正則表現へ(1)

例 $\Sigma = \{a, b\}$とする． 以下の言語は正則言語であるか?

$L_1 = \{ w \in \Sigma^+ \mid w$はabaで始まる文字列$\}$

$L_2 = \{ w \in \Sigma^+ \mid w$はabaで終わる文字列$\}$

$L_3 = \{ w \in \Sigma^+ \mid w$にabaが部分列(substring)として現れる文字列$\}$

オートマトンから正則表現へ(1)

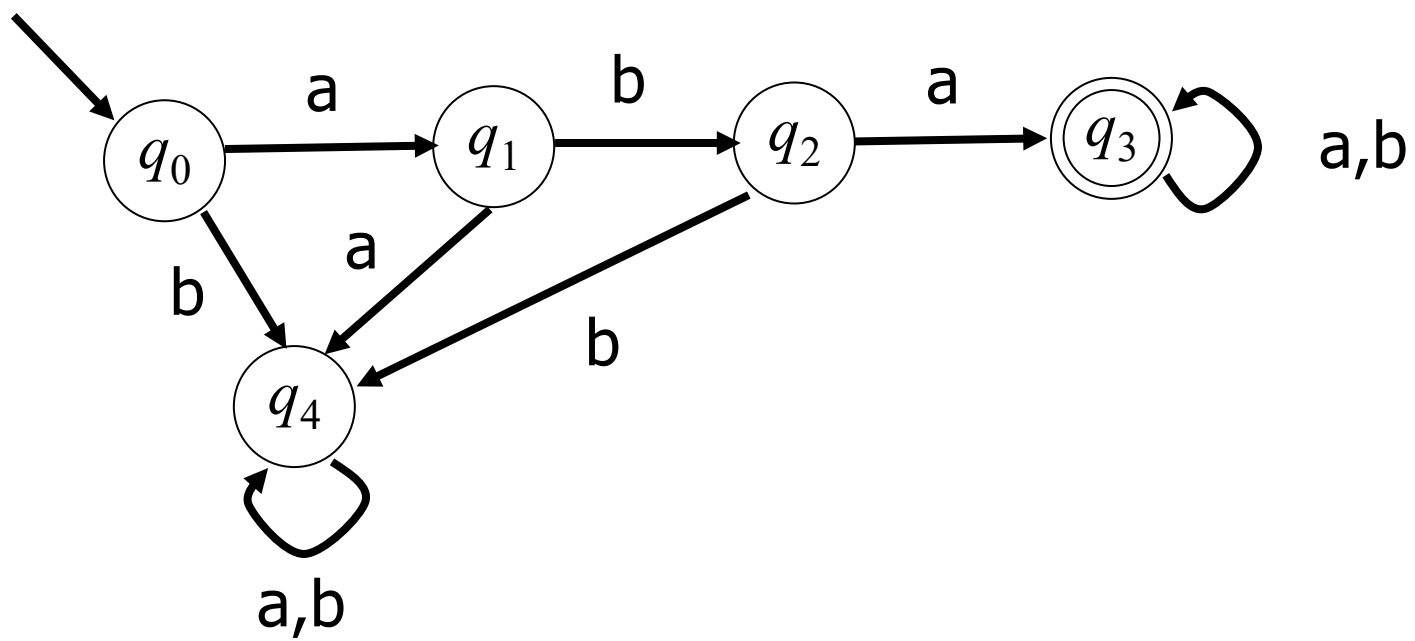
例 $\Sigma = \{a, b\}$とする．以下の言語は正則言語であるか?

$L_1 = \{ w \in \Sigma^+ \mid w$はabaで始まる文字列$\}$

$L_2 = \{ w \in \Sigma^+ \mid w$はabaで終わる文字列$\}$

$L_3 = \{ w \in \Sigma^+ \mid w$にabaが部分列(substring)として現れる文字列$\}$

M_1



4

オートマトンから正則表現へ(1)

例 $\Sigma = \{a, b\}$とする． 以下の言語は正則言語であるか?

$L_1 = \{ w \in \Sigma^+ \mid w$はabaで始まる文字列$\}$

$L_2 = \{ w \in \Sigma^+ \mid w$はabaで終わる文字列$\}$

$L_3 = \{ w \in \Sigma^+ \mid w$にabaが部分列(substring)として現れる文字列$\}$

5

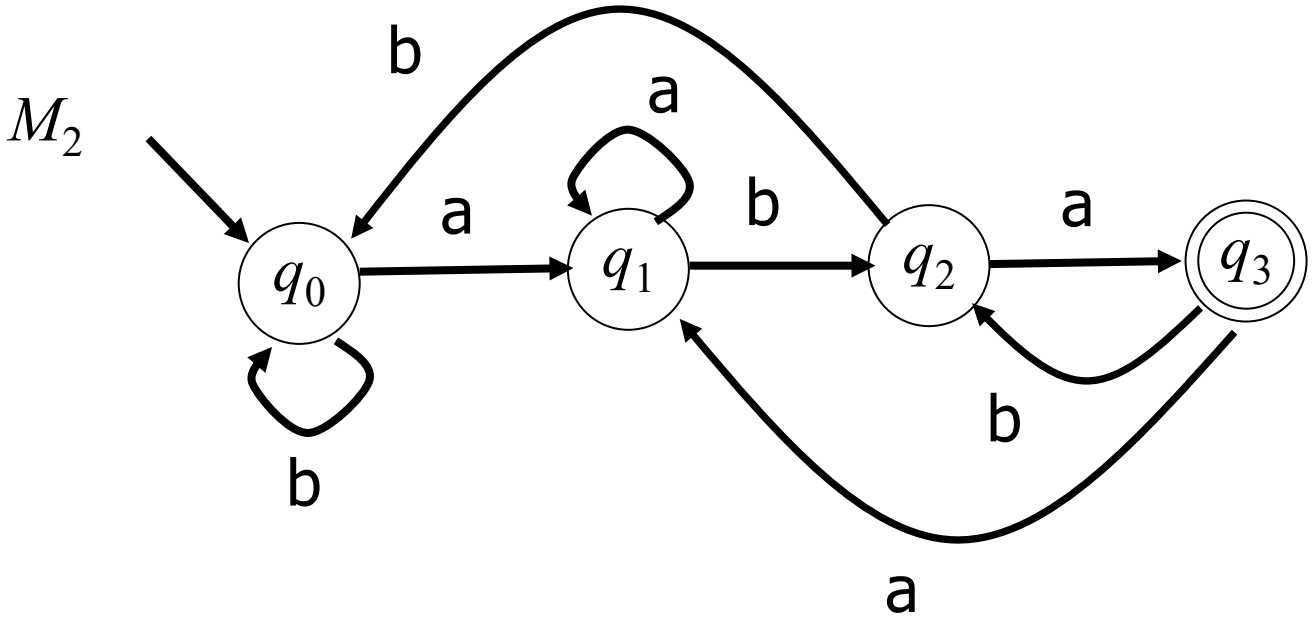
オートマトンから正則表現へ(1)

例 $\Sigma = \{a, b\}$とする．以下の言語は正則言語であるか?

$L_1 = \{ w \in \Sigma^+ \mid w$はabaで始まる文字列$\}$

$L_2 = \{ w \in \Sigma^+ \mid w$はabaで終わる文字列$\}$

$L_3 = \{ w \in \Sigma^+ \mid w$にabaが部分列(substring)として現れる文字列$\}$



6

オートマトンから正則表現へ(1)

例 $\Sigma = \{a, b\}$とする． 以下の言語は正則言語であるか?

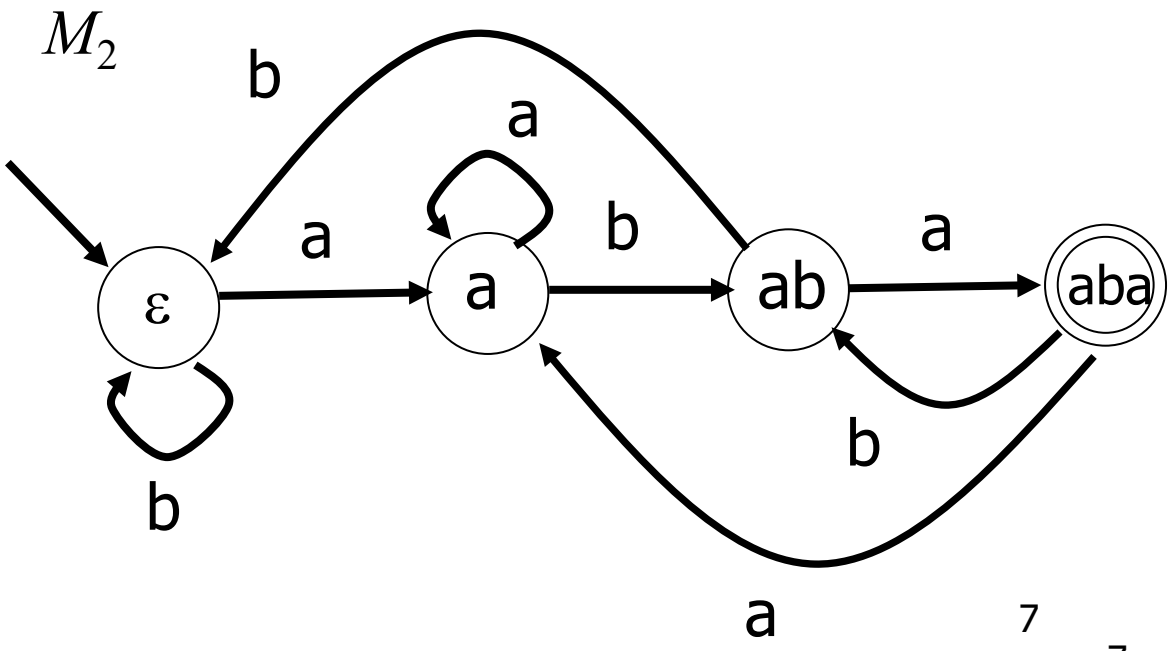
$L_1 = \{ w \in \Sigma^+ \mid w$はabaで始まる文字列$\}$

$L_2 = \{ w \in \Sigma^+ \mid w$はabaで終わる文字列$\}$

$L_3 = \{ w \in \Sigma^+ \mid w$にabaが部分列(substring)として現れる文字列$\}$

状態の「意味」

abaの
前綴り:ε, a, ab, aba
後綴り:aba, ba, a, ε



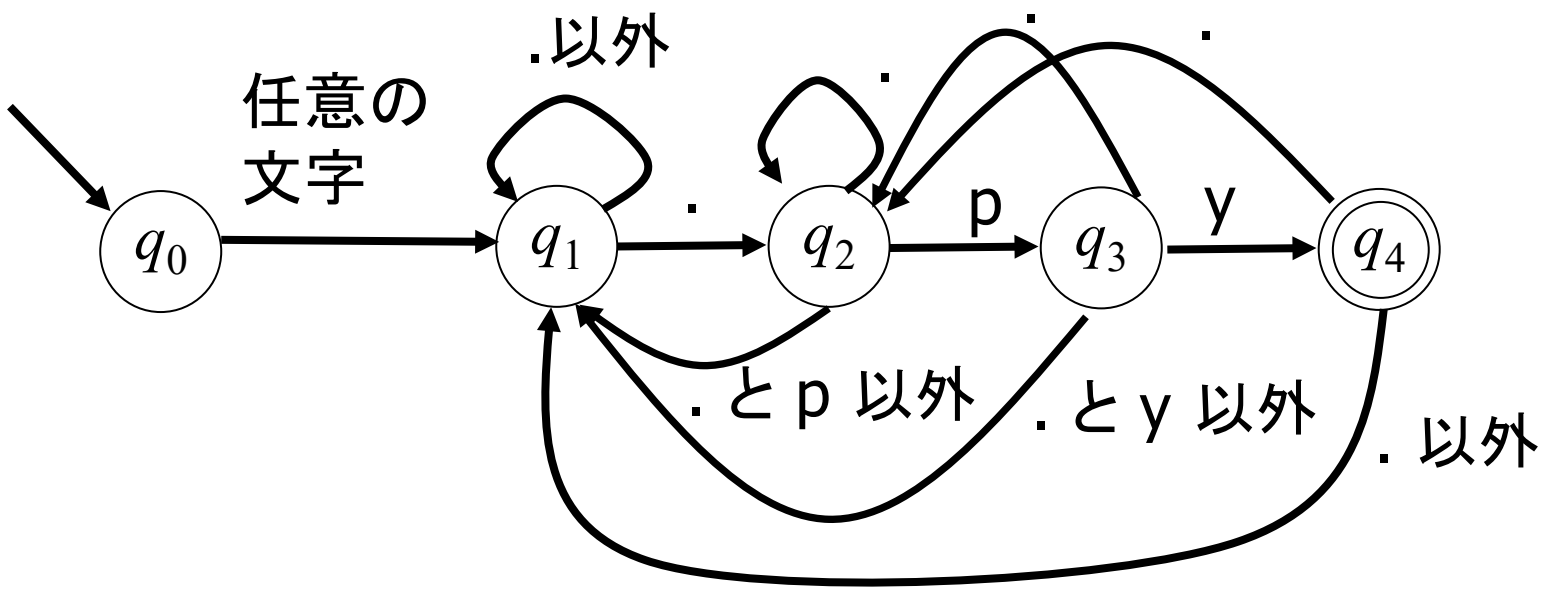
7

オートマトンから正則表現へ(2)

例 Python言語で記述されたプログラムのファイル名は
必ず　.py で終わり，その前に1文字以上を用いる
例えば

sample.py　　main.py

有限状態オートマトンで表現すると



オートマトンから正則表現へ(3)

例 HTMLで記述されたホームページのファイル名は

　必ず .html または .htm で終わり，その前に1文字以上を用いる．例えば

　　kyoto.html　　main.htm

正則表現(正規表現)とは

正則表現(regular expression)

- 計算機用語としては「正規表現」とよばれることもある
- パターンと呼ばれることも多い

- もともとは数学者 S. Kleene がオートマトンの動作を表記する方法として考案
- Linux, UbuntuなどUNIX系のshellやWindowsのコマンドライン

```
$ ls *.py
$ ls [abc]*.py
```

- Perlなどのプログラミング言語やスクリプト言語やでも正則表現を導入

```
/ab.cd/
/ab*cd/
```

- “文法”は言語やシステムによってまちまち

Σ上の正則表現

Σ^*の部分集合(形式言語)を表すための式(expression)

正則表現 $\boldsymbol{R}$ が表す言語を$L(\boldsymbol{R})$と表す.

1. $\emptyset$ (太字)は (Σ上の)正則表現である. $L(\emptyset)=\{\}$ と定義する.
2. $\boldsymbol{\varepsilon}$ (太字)は正則表現である. $L(\boldsymbol{\varepsilon})=\{\varepsilon\}$と定義する.
3. $c\in\Sigma$ に対して $\boldsymbol{c}$ は正則表現である. $L(\boldsymbol{c})=\{\ c\ \}$と定義する.
4. $\boldsymbol{R}, \boldsymbol{S}$ は正則表現であり, $L(\boldsymbol{R})=R$, $L(\boldsymbol{S})=S$ と定義されているとき,
   - $(\boldsymbol{R}+\boldsymbol{S})$は正則表現であり，$L((\boldsymbol{R}+\boldsymbol{S}))=R\cup S$ と定義する.
   - $(\boldsymbol{RS})$は正則表現であり，$L((\boldsymbol{RS}))=RS$と定義する.
   - $(\boldsymbol{R}^*)$は正則表現であり，$L((\boldsymbol{R}^*))=R^*$ を表す正則表現である
5. 上の1から3から開始して，4を有限回適用して得られる式だけが正則表現である.

形式言語のKleene閉包

- 形式言語 L に対して次のように定義される集合L^*をLのKleene閉包もしくは単に閉包という

$$L^0 = \{\varepsilon\} \quad (\text{空集合ではない!})$$

$$L^n = L\,L^n = \{\, uv \mid u \in L, v \in L^n \,\}\ (n \geq 1)$$

$$L^* = \{\varepsilon\} \cup L \cup L^2 \cup L^3 \cup \ldots = \bigcup_{n=1}^{\infty} L^n$$

Example

L ={aa, ab}とすると

L^2 ={aaaa, aaab, abaa, abab}

L^3 ={aaaaaa, aaaaab, aaabaa, aaabab, abaaaa,...}

...

L^* ={ε, aa, ab, aaaa, aaab, abaa, abab, aaaaaa,...}

Σ上の正則表現の例(1)

Σ={a, b}とする

1. ∅ (太字)は (Σ上の)正則表現である. $L(\emptyset)=\{\}$ と定義する.
2. **ε** (太字)は正則表現である. $L(\boldsymbol{\varepsilon})=\{\varepsilon\}$と定義する.
3. $c \in \Sigma$ に対して $\boldsymbol{c}$ は正則表現である. $L(\boldsymbol{c})=\{\ c\ \}$と定義する.

規則3から, **a**, **b**は正則表現であり, $L(\mathbf{a})=\{\ \mathrm{a}\ \}$, $L(\mathbf{b})=\{\ \mathrm{b}\ \}$である.

Σ上の正則表現の例(2)

規則3から, **a**, **b**は正則表現であり, $L(\mathbf{a})=\{\,\mathrm{a}\,\}$, $L(\mathbf{b})=\{\,\mathrm{b}\,\}$である.

4. $\boldsymbol{R}, \boldsymbol{S}$ が正則表現であり, $L(\boldsymbol{R})=R$, $L(\boldsymbol{S})=S$ と定義されているとき,
   - $(\boldsymbol{R}+\boldsymbol{S})$は正則表現であり, $L((\boldsymbol{R}+\boldsymbol{S}))=R\cup S$ と定義する.

この規則から

(a+a), (a+b), ((a+a)+a), ..., ((a+b)+(b+b)), ...

などは全て正則表現である. これらが表す言語はそれぞれ

$L(\mathbf{(a+a)})=\{\mathrm{a}\}\cup\{\mathrm{a}\}=\{\mathrm{a}\}$

$L(\mathbf{(a+b)})=\{\mathrm{a}\}\cup\{\mathrm{b}\}=\{\mathrm{a},\mathrm{b}\}$

$L(\mathbf{((a+a)+a)})=L(\mathbf{(a+a)})\cup L(\mathbf{(a)})$

$=(L(\mathbf{a})\cup L(\mathbf{a}))\cup L(\mathbf{(a)})=(\{\mathrm{a}\}\cup\{\mathrm{a}\})\cup\{\mathrm{a}\}=\{\mathrm{a}\}$

$L(\mathbf{((a+b)+(b+b))})=L(\mathbf{(a+b)})\cup L(\mathbf{(b+b)})$

$=(L(\mathbf{a})\cup L(\mathbf{b}))\cup(L(\mathbf{b})\cup L(\mathbf{b}))=(\{\mathrm{a}\}\cup\{\mathrm{b}\})\cup(\{\mathrm{b}\}\cup\{\mathrm{b}\})=\{\mathrm{a},\mathrm{b}\}$

Σ上の正則表現の例(3)

4. $\boldsymbol{R}, \boldsymbol{S}$ は正則表現であり, $L(\boldsymbol{R})=R$, $L(\boldsymbol{S})=S$ と定義されているとき,
   - $(\boldsymbol{R}+\boldsymbol{S})$は正則表現であり，$L((\boldsymbol{R}+\boldsymbol{S}))=R \cup S$ と定義する.
   - $(\boldsymbol{RS})$は正則表現であり，$L((\boldsymbol{RS}))= RS$と定義する.
   - $(\boldsymbol{R}^*)$は正則表現であり，$L((\boldsymbol{R}^*)) = R^*$ を表す正則表現である

これらの規則から

a+a,..., **aa**, **ab**, **ba**, **bb**, **a***, **b***,

((a+a)+a),..., **((aa)a)**,…, **(a(ba))**,…, **(a*)*** , **(b*)*** ,

(a+(bb)), **((ab)+b)**, ..., **(ab)***,...

などは全て正則表現である. これらが表す言語はそれぞれ

$L((\mathbf{aa})) = \{a\}\{a\} = \{aa\}$, $L((\mathbf{ab})) = \{a\}\{b\} = \{ab\}$, …

$L((\mathbf{aa})\ \mathbf{a}) = L((\mathbf{aa})) \cup L(\mathbf{a}) = \{aa\}\{a\} = \{aaa\}$, …

のように帰納的に計算する.

Σ上の正則表現に関する注意(1)

- 正則表現中の括弧 (,) については，演算子の結合力の強さを

  $(R^*) > (RS) > (R + S)$

  と仮定して(曖昧さが生じない限り)省略するのが一般的である.

  ((ab)+b) ⇨ **ab+b**

  ((ab)+(b)*) ⇨ **ab+b***

  ((ab)+(bb))* ⇨ **(ab+bb)***

  - 正則表現が表す言語に対する

    結合律 $L(((R+S)+T))=L((R+(S+T)))$, $L(((RS)T))=L((R(ST)))$

    冪等律 $L(((R^*)^*))= L((R^*))$

    が成立するから,同じ演算子の繰り返しも省略する.

Σ上の正則表現の例

Σ={a, b}とする

- $L(\mathbf{a(a+b)^*b}) = L(\mathbf{a(\Sigma^*)b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ $u \in \Sigma^* \}$
- $L(\mathbf{a(ab)^*b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ $u \in \{\mathrm{ab}\}^*\}$
  $= \{\mathrm{ab}, \mathrm{aabb}, \mathrm{aababb}, \ldots\}$
- $L(\mathbf{a(aa+ab)^*b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ $u \in (\{\mathrm{ab}\} \cup \{\mathrm{ab}\}^*)\}$
  $= \{\mathrm{ab}, \mathrm{aaab}, \mathrm{aabb}, \mathrm{aaaaab}, \mathrm{aaaabb}, \ldots\}$

Σ上の正則表現に関する注意(2)

5. 上の1から3から開始して，4を有限回適用して得られる式だけが正則表現である.

この条件は，正則表現が帰納的に定義される，ことを表している.

- **(a-b)** や **(a+b)+** などは正則表現ではない.

(参照) 数学的帰納法は，自然数に関する性質(公理)が根拠

I. 0(または1)は自然数である.

II. nが自然数であれば，$n+1$も自然数である.

III. I, IIで自然数とわかるものだけが自然数である.

(注)IIIがなければ，次のような集合が自然数全体の集合ではないとはいえなくなる

$\{-1, 0, 1, 2, 3, 4, 5,...\}$

$\{0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5,...\}$

Σ上の正則表現の例(1)

Σ={a, b}とする.

1. ∅ (太字)は空集合{ } を表す(Σ上の)正則表現である
2. ε (太字)は集合{ ε } を表す正則表現である
3. $c \in \Sigma$ に対して $\boldsymbol{c}$ は集合{ c } を表す正則表現である

上の3つの規則より,

1. $\boldsymbol{R}, \boldsymbol{S}$ がΣ^*の部分集合 R, S を表す正則表現であるとき
   - $(\boldsymbol{R} + \boldsymbol{S})$ は集合 $R \cup S$ を表す正則表現である
   - $(\boldsymbol{RS})$ は集合 RS を表す正則表現である
   - $(\boldsymbol{R}^*)$ は集合 R^* を表す正則表現である
2. 上の1から3から開始して, 4を有限回適用して得られる式だけが正則表現である.

正則表現が表す言語

- 正則表現 $\boldsymbol{R}$ が表す言語を$L(\boldsymbol{R})$と表す.

Examples

$L(\mathbf{ab}) = \{\mathrm{ab}\}$

$L(\mathbf{a(a+b)^*b}) = L(\mathbf{a(\Sigma^*)b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ $u \in \Sigma^* \}$

$L(\mathbf{a(ab)^*b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ $u \in \{\mathrm{ab}\}^*\}$

$= \{\mathrm{ab}, \mathrm{aabb}, \mathrm{aababb}, \ldots\}$

$L(\mathbf{a(aa+ab)^*b}) = \{w \mid w = \mathrm{a}u\mathrm{b}$ かつ$u \in \{\{\mathrm{aa}\} \cup \{\mathrm{ab}\}\}^*\}$

$= \{\mathrm{aaab}, \mathrm{aabb}, \mathrm{aaaaab}, \mathrm{aaaabb}, \mathrm{aabaab},$

$\mathrm{aababb}, \mathrm{aaaaaaab}, \ldots\}$

正則表現の等価性

- 正則表現 R, S について$L(R)=L(S)$のとき, RとSは等価であるといい，$R = S$ と表す.

Example

$(a+b)^* = ((a^*)(b^*))^*$

公式集

$\varnothing^* = \varepsilon \qquad \varepsilon R = R\varepsilon = R \qquad R + R = R \qquad R + S = S + R$

$(R + S)T = ST + RT \qquad T(R + S) = TS + TR$

$(R^*)^* = R^* \qquad (R^*)(R^*)^* = R^* \qquad R^*R + \varepsilon = R^*$

$(R^*S^*)^* = (R + S)^* \qquad (R^*S)^*R^* = (R + S)^*$

$(R^*S)^* + (S^*R) = (R + S)^* \qquad (R + S)^*S + \varepsilon = (R^*S)^*$

$R^*S + S = R^*S \qquad RS^* + R = RS^*$

Σ上の正則表現(補)

- アルファベットΣを表すΣ (太字)を用いることもある.
  - $\Sigma=\{c_1,\ldots, c_n\}$とすると$\boldsymbol{\Sigma} = \boldsymbol{c}_1+\cdots+\boldsymbol{c}_n$
- UNIX系のshellやWindowsのコマンドライン, Python などのプログラミング言語では

  $\boldsymbol{R}+\boldsymbol{S}$ ではなく $\boldsymbol{R}\,|\,\boldsymbol{S}$ で $R \cup S$ を表す

  $\boldsymbol{R}^+$ で $R(R*)$を表す
  - コマンドラインやプログラミング言語では上添え字が使えない. そこで, * だけで $\boldsymbol{\Sigma}^*$ を表し, + だけで $\boldsymbol{\Sigma}^+=\boldsymbol{\Sigma}$ を表す.

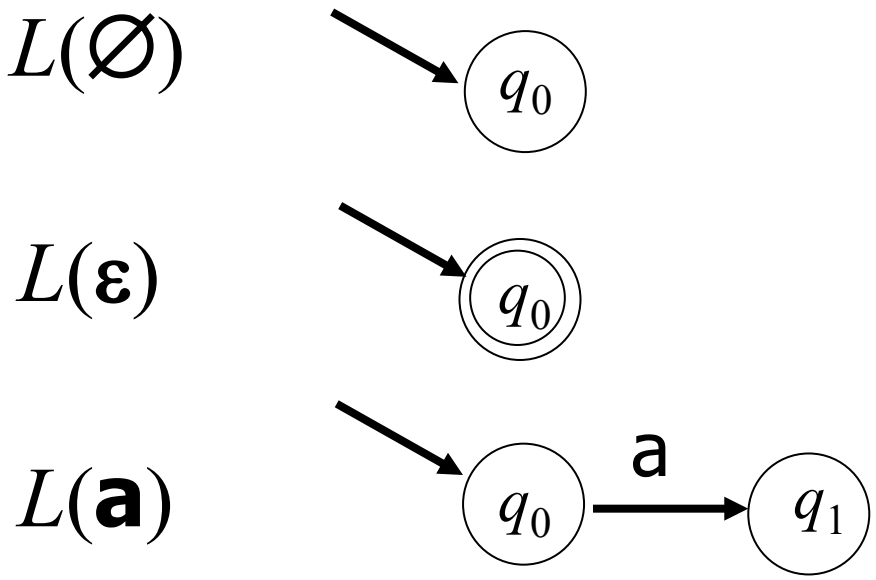


正則表現から有限状態オートマトンへの変換

基本的なアイデア(1)

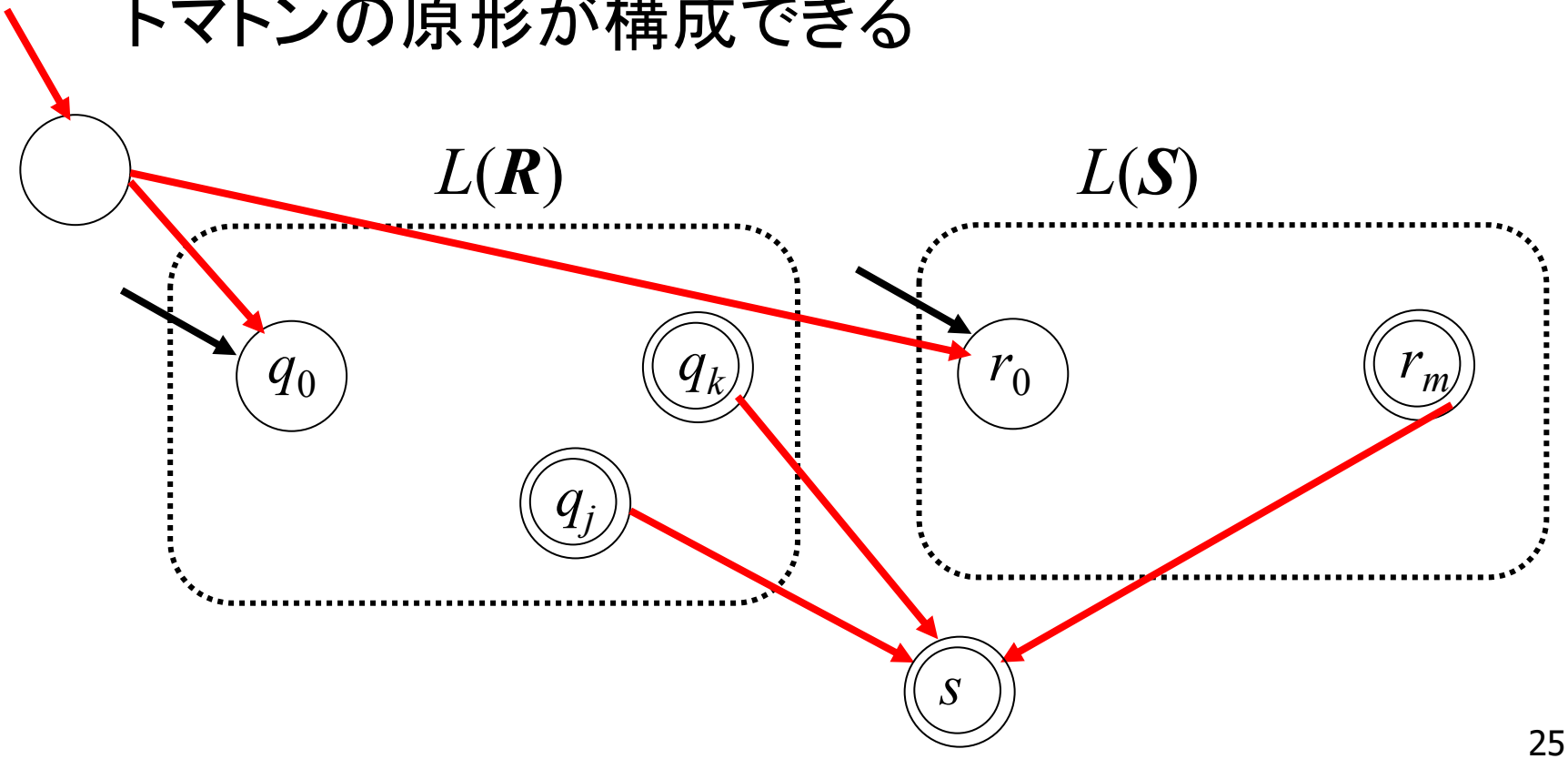
- 正則表現∅ , ε, c に対して$L(\emptyset)$, $L(\varepsilon)$, $L(c)$を受理する有限状態オートマトンの原形は構成できる

$\Sigma = \{a, b\}$のとき



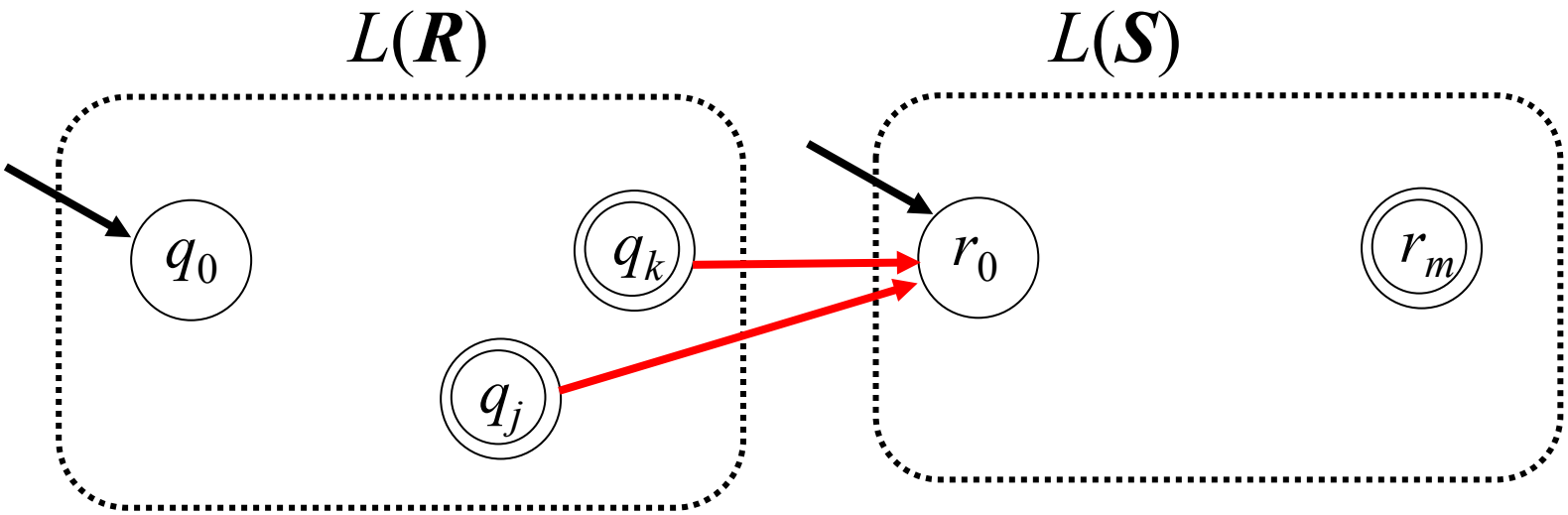
基本的なアイデア(2)

- 正則表現$\boldsymbol{R}$, $\boldsymbol{S}$ に対して$L(\boldsymbol{R})$, $L(\boldsymbol{S})$を受理する有限状態オートマトンの一部が構成できていれば，それらを“並列”に繋げば，$L(\boldsymbol{R}+\boldsymbol{S})$を受理する有限状態オートマトンの原形が構成できる



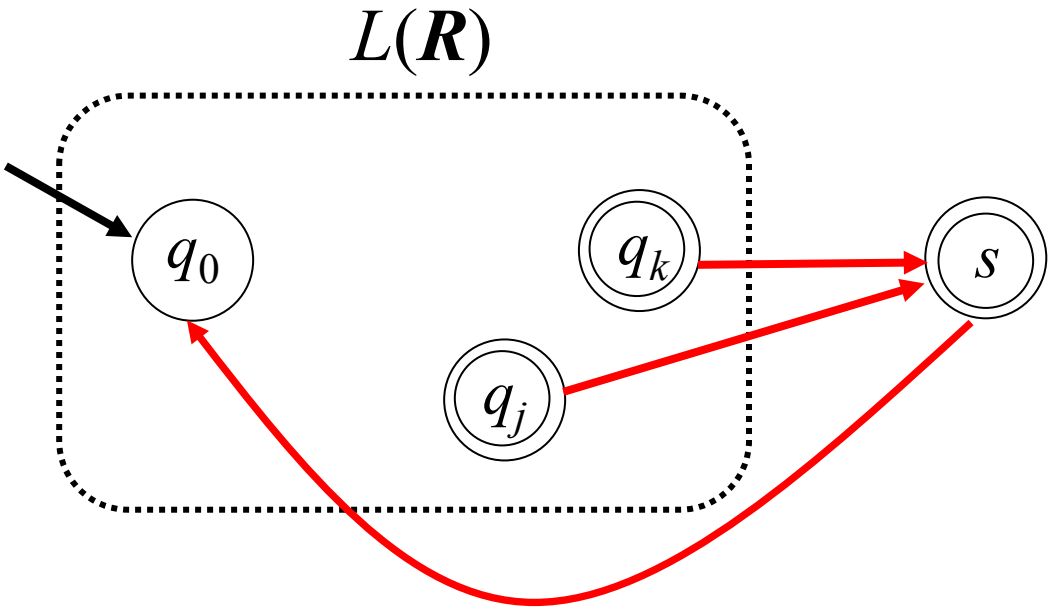
基本的なアイデア(3)

- 正則表現$\boldsymbol{R}$, $\boldsymbol{S}$ に対して$L(\boldsymbol{R})$, $L(\boldsymbol{S})$を受理する有限状態オートマトンの原形が構成できていれば，それらを“直列”に繋げば，$L(\boldsymbol{RS})$を受理する有限状態オートマトンの原形が構成できる



基本的なアイデア(3)

- 正則表現$\boldsymbol{R}$に対して$L(\boldsymbol{R})$を受理する有限状態オートマトンの原形が構成できていれば，それを用いて“ループ”を構成すれば，$L(\boldsymbol{R}^*)$を受理する有限状態オートマトンの原形が構成できる

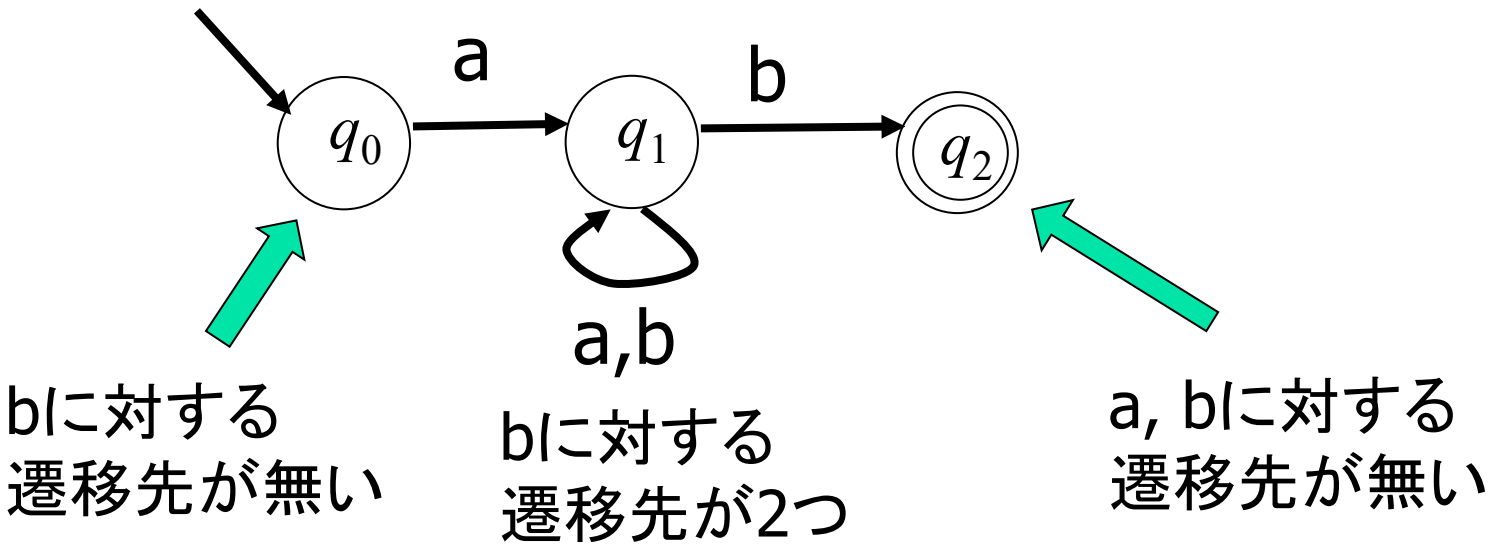


非決定性有限状態オートマトン

- 状態q_iにおいて記号cが読み込まれたときの遷移先が0個以上

Example

$L(\mathbf{a(\Sigma^*)b}) = \{ w \in \Sigma^* \mid w$はaで始まりbで終わる$\}$



非決定性有限状態オートマトン

- 以下のような構成要素からなる組$M=(\Sigma, S, \delta, q_0, F)$を非決定性有限状態オートマトン(NFA)とよぶ.
  - Σ はアルファベット
  - S は空でない集合であり,
    その要素を状態とよぶ.
  - δ は $S\times\ \Sigma \rightarrow 2^S$ なる関数
    - δ は右のような表で表現される

	F	a_1	…	a_n
q_0				
…				
q_m				

  - $q_0 \in S$ は特定の状態であり, 初期状態とよぶ.
  - $F \subset S$は特定の状態の集合であり, その要素を終了状態, もしくは受理状態とよぶ.

非決定性有限状態オートマトンが受理する記号列

- 非決定性有限状態オートマトン$M = (\Sigma, S, \delta, q_0, F)$
- 記号列$w = c_1 c_2 \cdots c_k$ に対して

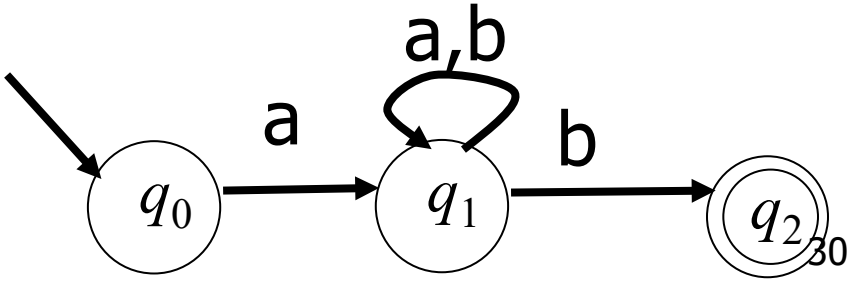
  $\delta(q_0, c_1) \ni q_1, \delta(q_1, c_2) \ni q_2, \ldots, \delta(q_{k-1}, c_k) \ni q_k$ かつ $q_k \in F$ を満たす状態の列$q_1, q_2 \ldots, q_k$を構成できるとき，Mはwを受理するという

Example

abb ： $\delta(q_0, \mathrm{a}) \ni q_1, \delta(q_1, \mathrm{b}) \ni q_1, \delta(q_1, \mathrm{b}) \ni q_2$ ， $q_2 \in F$

aaa ： $\delta(q_0, \mathrm{a}) \ni q_1, \delta(q_1, \mathrm{a}) \ni q_1, \delta(q_1, \mathrm{a}) \ni q_1$ ， $q_2 \notin F$

b ： $\delta(q_0, \mathrm{b}) = \varnothing$

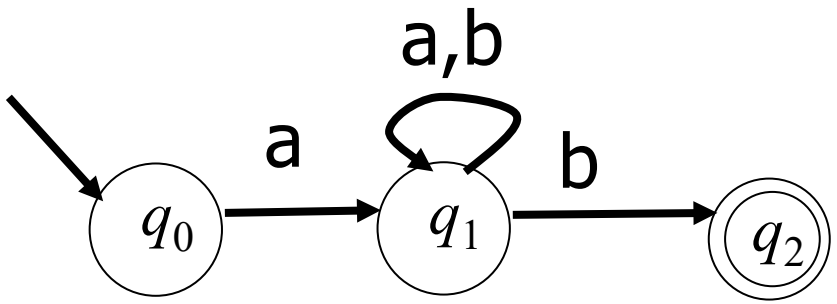


非決定性有限状態オートマトンが受理する言語

- 非決定性有限状態オートマトン$M = (\Sigma, S, \delta, q_0, F)$に対して

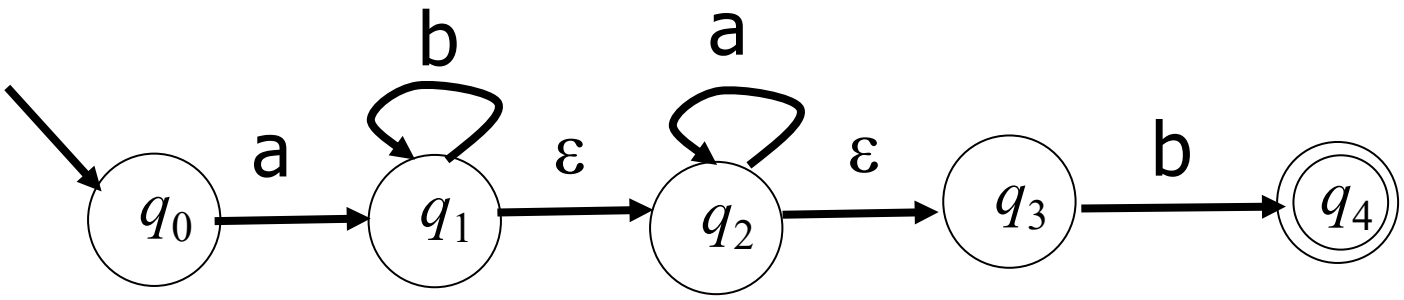
  $L(M) = \{w \in \Sigma^* \mid w$は$M$によって受理される$\}$

  をMが受理する言語という.



ε遷移

- 状態qにおいて記号が読み込まれなくても状態を遷移することを認めるとき, そのような遷移をε遷移という.
  - 記号を読む前後どちらでも好きなだけ遷移してよい



εを持つ非決定性有限状態オートマトン

- 以下のような構成要素からなる組$M = (\Sigma, S, \delta, q_0, F)$をε遷移を持つ非決定性有限状態オートマトンとよぶ.
  - Σ はアルファベット
  - S は空でない集合であり，その要素を状態とよぶ.
  - δ は $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^S$ なる関数
    - δ は右のような表で表現される
  - $q_0 \in S$ は特定の状態であり，初期状態とよぶ.
  - $F \subset S$は特定の状態の集合であり，その要素を終了状態，もしくは受理状態とよぶ.

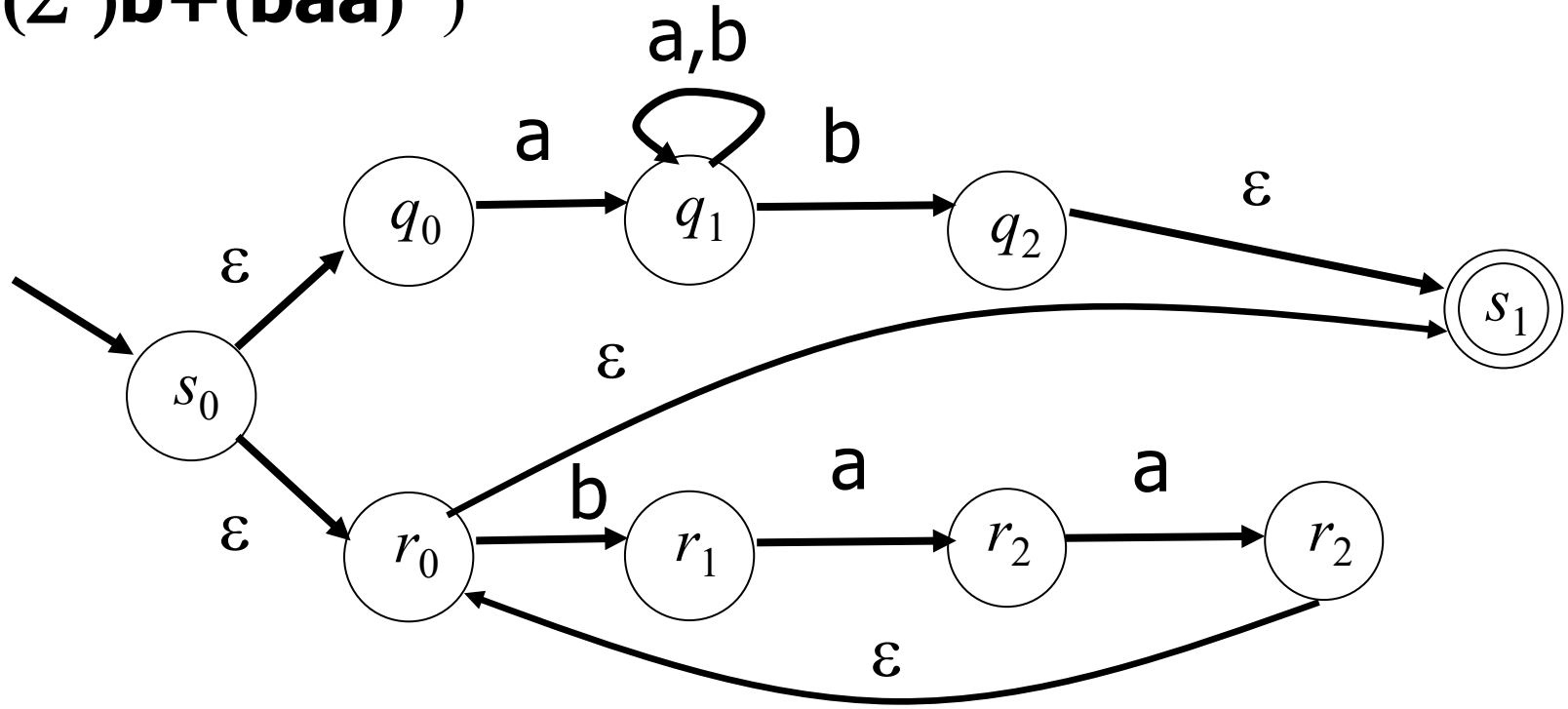
	F	a_1	…	a_n	ε
q_0					
…					
q_m					

正則表現とオートマトン

- 正則表現をε遷移を持つ非決定性有限状態オートマトンへ変換する

Example

$L(\mathbf{a}(\Sigma^*)\mathbf{b}+(\mathbf{baa})^*)$



NFAからDFAへの変換の方針

- 部分集合構成法

ε遷移を持つ非決定性オートマトン$M = (\Sigma, S, \delta, q_0, F)$

$$\delta: S \times (\Sigma \cup \{\varepsilon\}) \to 2^S$$



決定性オートマトン$M' = (\Sigma, 2^S, \delta', q_0, F)$

$$\delta': S \times \Sigma \to S$$

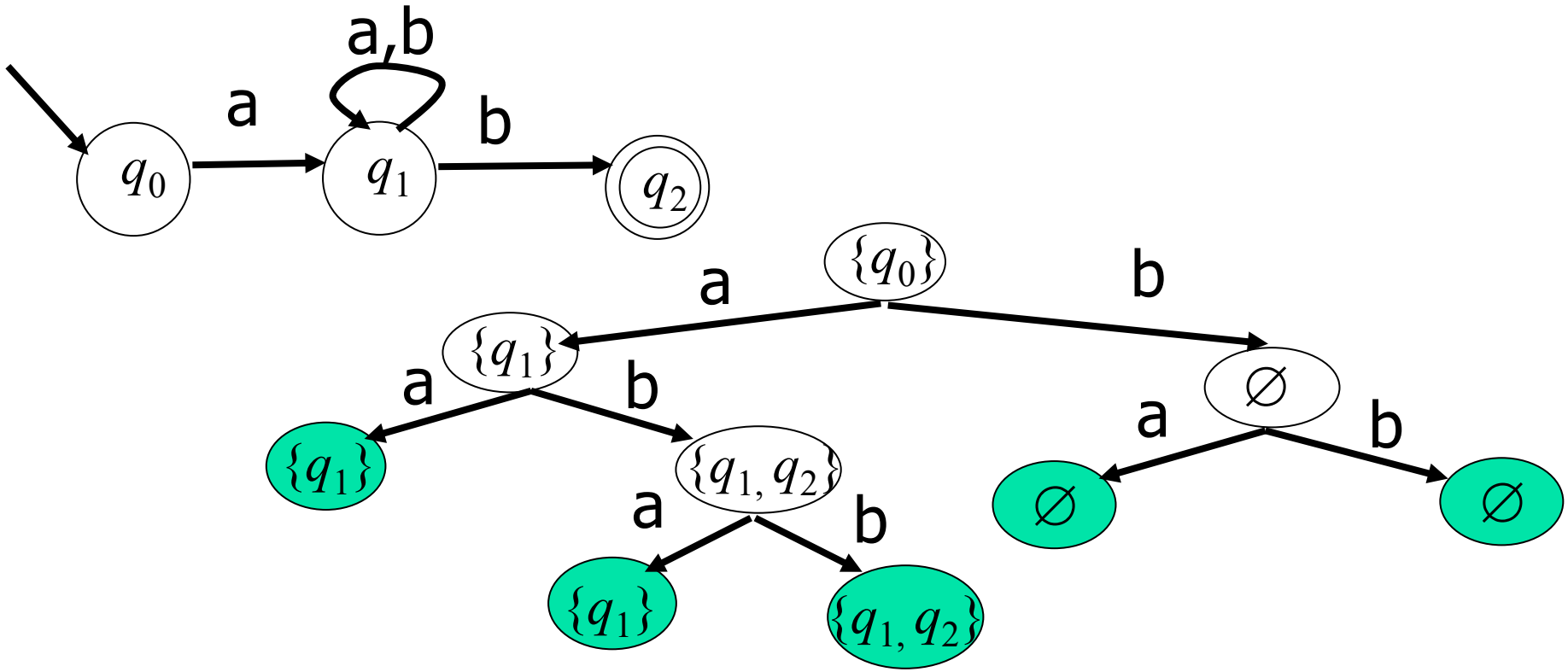
- 記法

非決定性オートマトンMの状態の集合$Q = \{q_{i1}, \ldots, q_{ik}\}$

$$\delta(Q, a) = \delta(q_{i1}, c) \cup \ldots \cup \delta(q_{i1}, c)$$

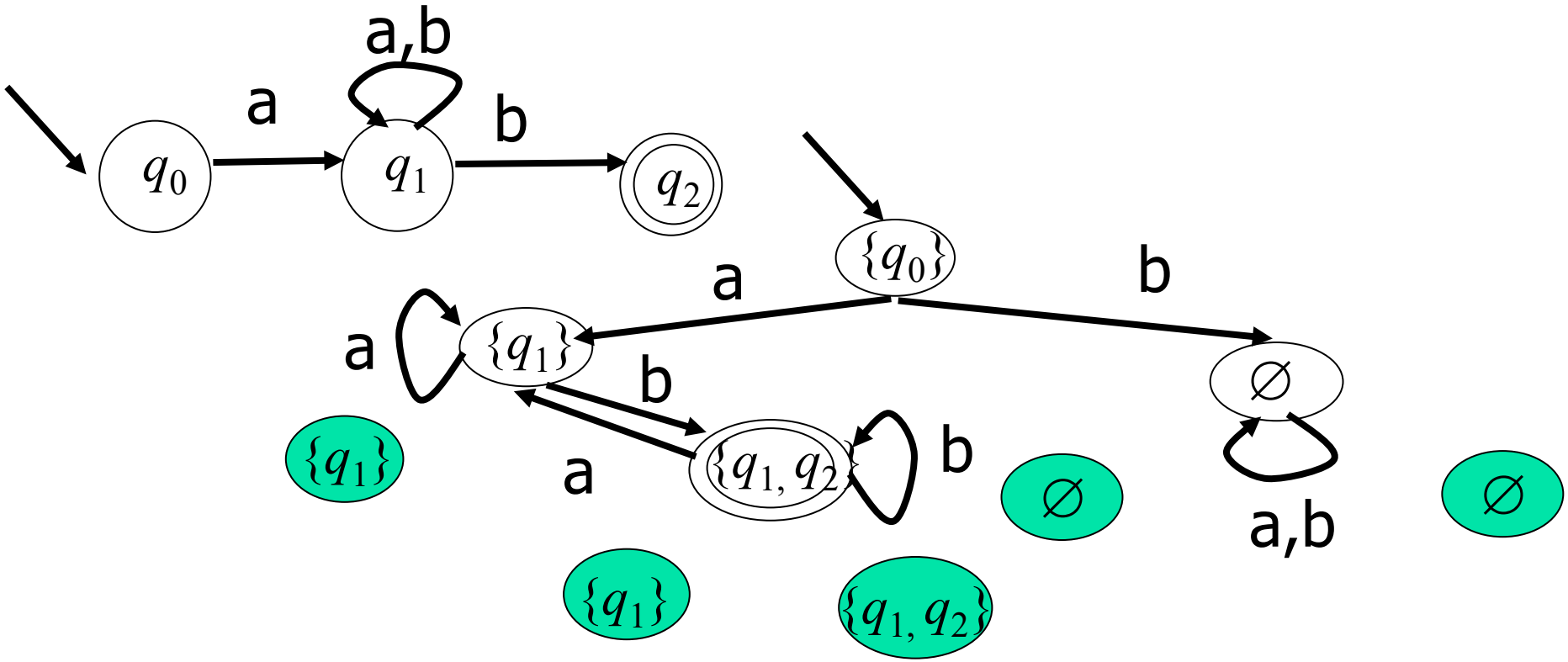
ε遷移が無い場合

- 根に$\{q_0\}$をラベル付けする. ラベルがQである節点の子に$\delta(Q, c)$をラベル付けする



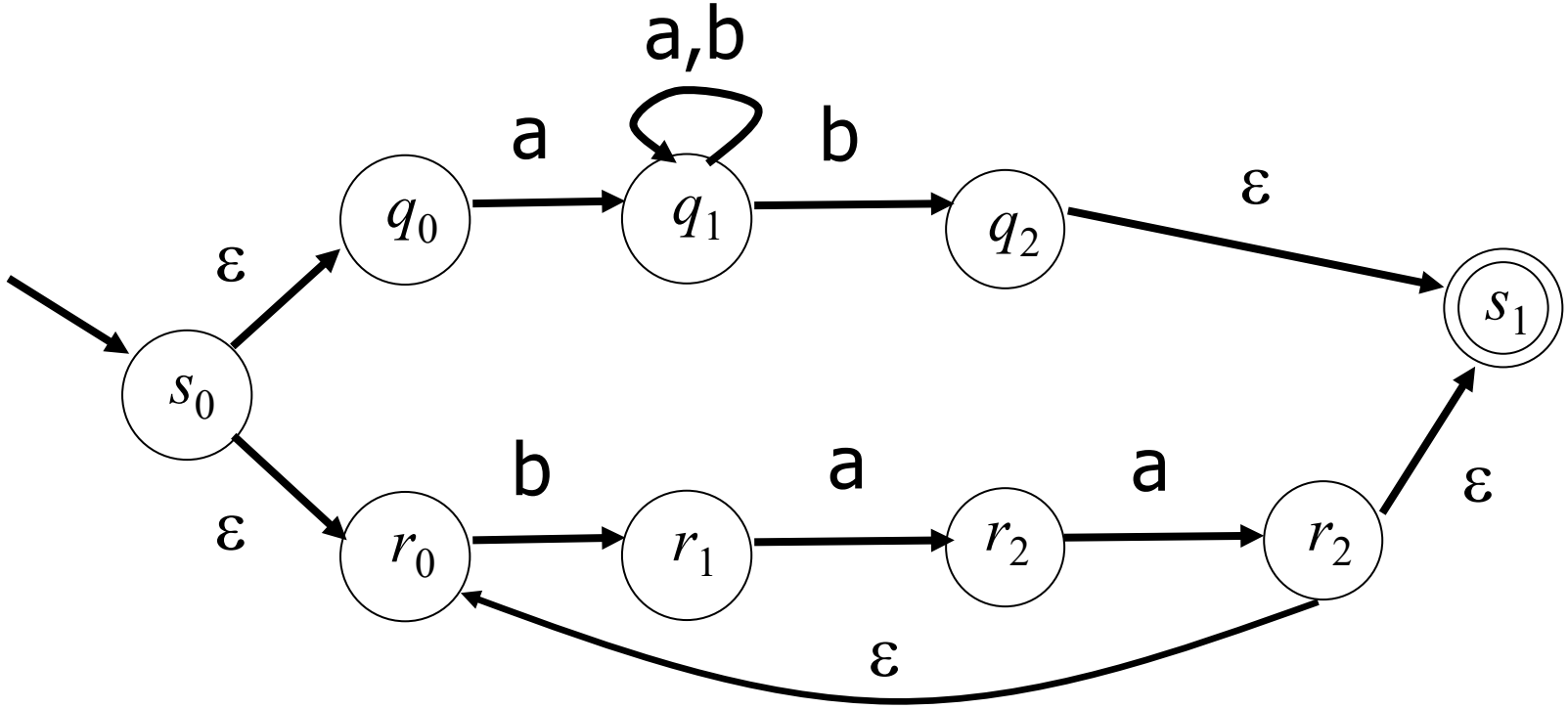
ε遷移が無い場合

- 同じラベルのノードを一つにして状態遷移とする
- 受理状態を含むノードを受理状態にする



ε閉包

- ε遷移を持つ非決定性有限状態オートマトン$M = (\Sigma, S, \delta, s_0, F)$において，状態$q$からεで遷移する先全体の集合($q$自身を含む)を$q$のε閉包とよび，ε-cl($q$)で表す．



ε遷移の除去

- ε遷移を持つ非決定性有限状態オートマトン$M = (\Sigma, S, \delta, s_0, F)$に対して,

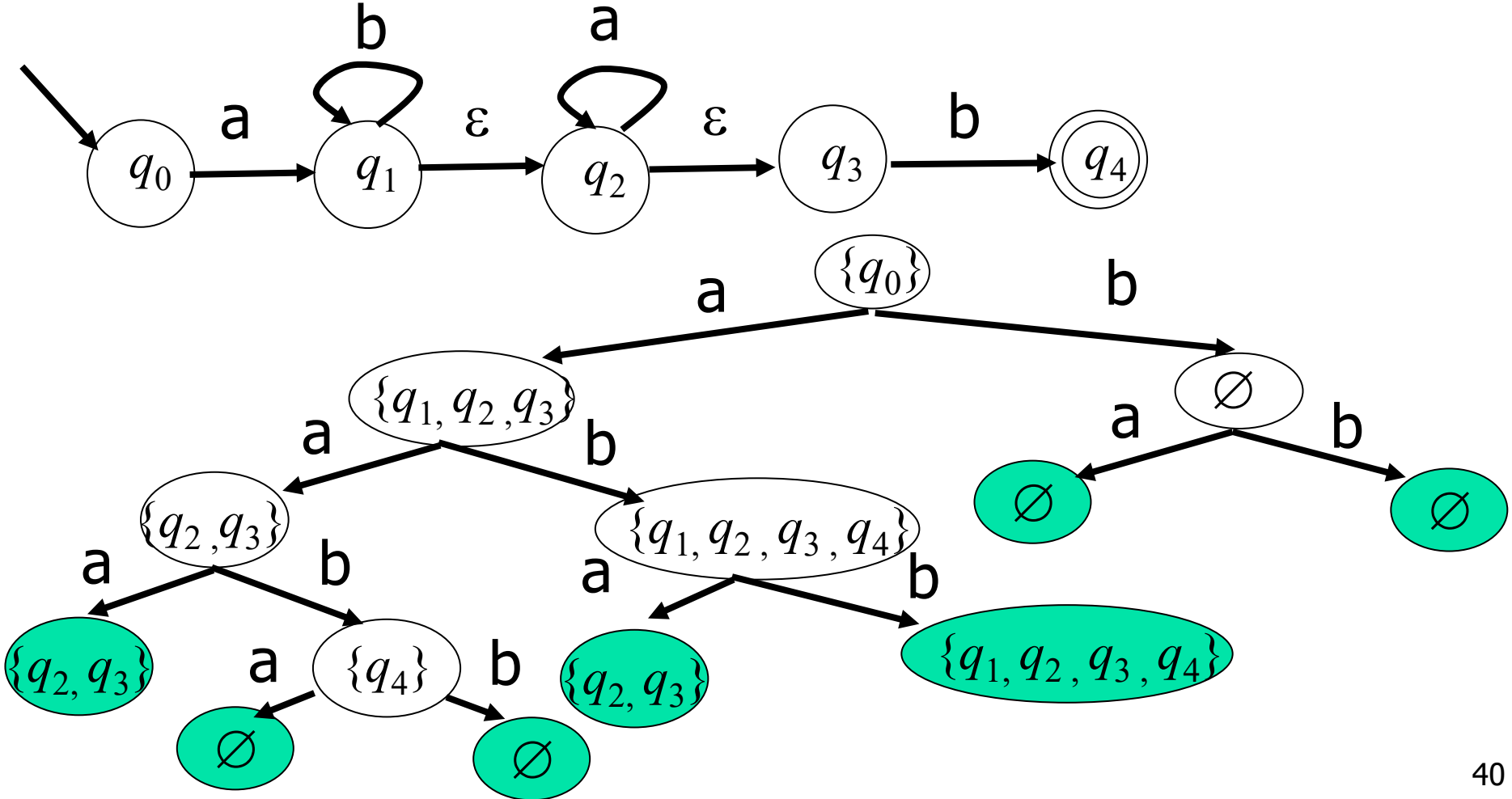
$$\delta'(q, c) = \varepsilon\text{-cl}(\delta(\varepsilon\text{-cl}(q), c)$$

と定義して得られる非決定性有限状態オートマトンを$M' = (\Sigma, S, \delta', q_0, F)$とすると$L(M) = L(M')$が成立つ.

  - 証明は記号列wの長さに関する帰納法による

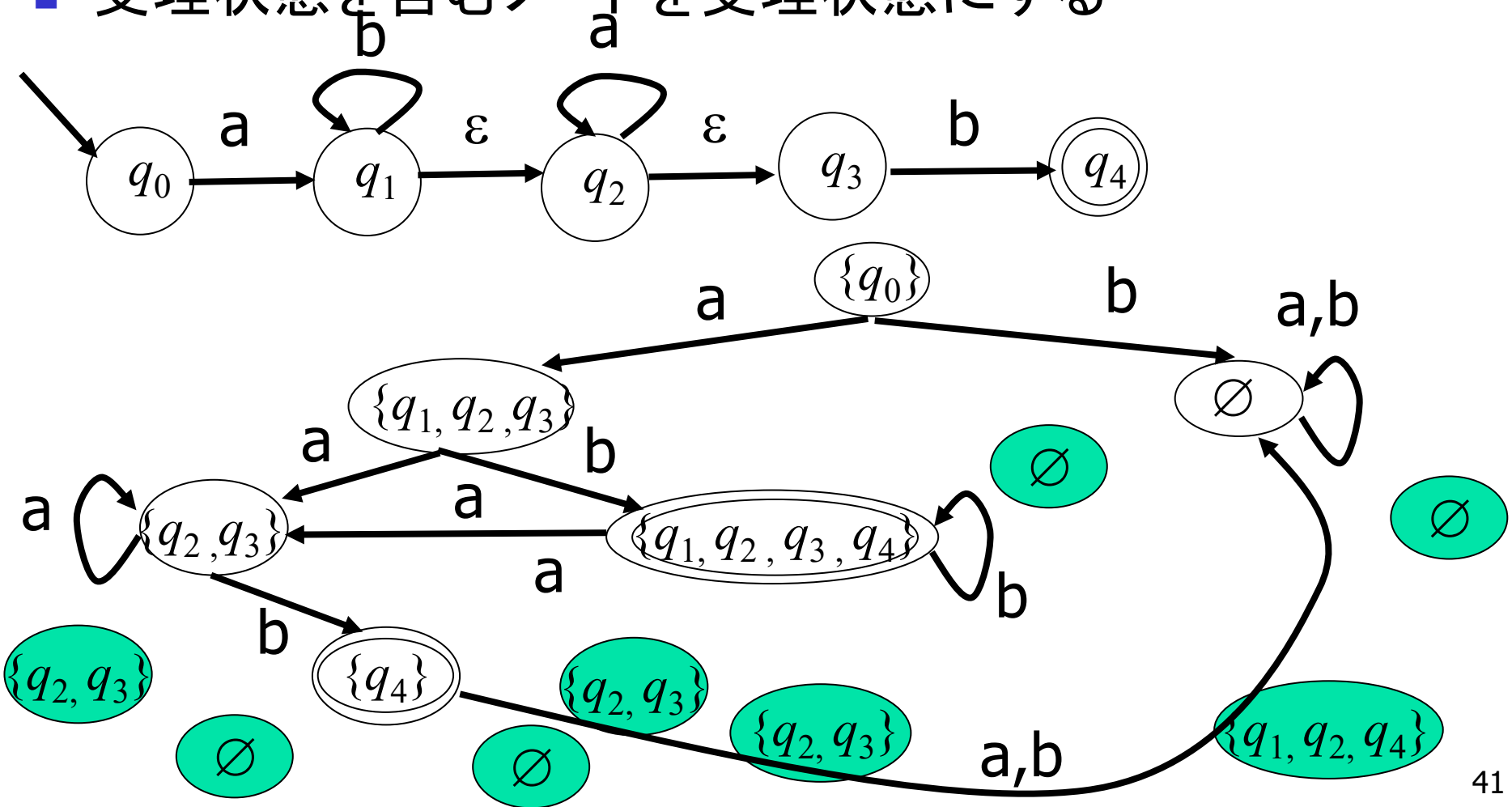
ε遷移がある場合

- 根にε-cl($\{q_0\}$)をラベル付けする. ラベルがQである節点の子に$\delta'(Q, c)$ = ε-cl(δ(ε-cl(Q), c) をラベル付けする



ε遷移がある場合

- 同じラベルのノードを一つにして状態遷移とする
- 受理状態を含むノードを受理状態にする

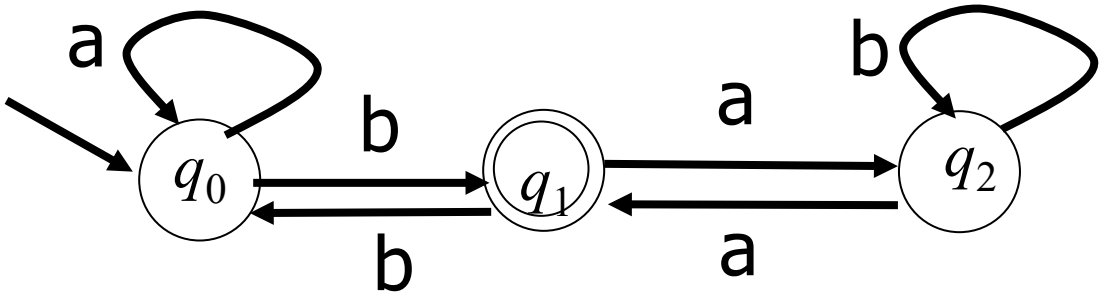




正則表現から有限状態オートマトンへの変換

McNorton-Yamadaの方法(1)

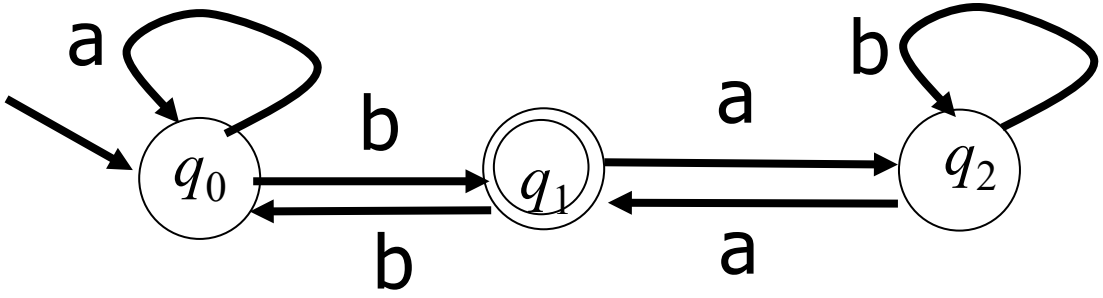
- Step -1. q_iからq_jへ直接遷移する記号だけからなる集合を正則表現で表す．ただし，i=jについてはεも加える



$$R_{00} = \mathbf{a} + \varepsilon,\ R_{11} = \varnothing + \varepsilon = \varepsilon,\ R_{22} = \mathbf{b} + \varepsilon,$$
$$R_{01} = \mathbf{b},\ R_{02} = \varnothing,\ R_{10} = \mathbf{b},\ R_{12} = \mathbf{a},$$
$$R_{20} = \varnothing,\ R_{21} = \mathbf{a},$$

McNorton-Yamadaの方法(2)

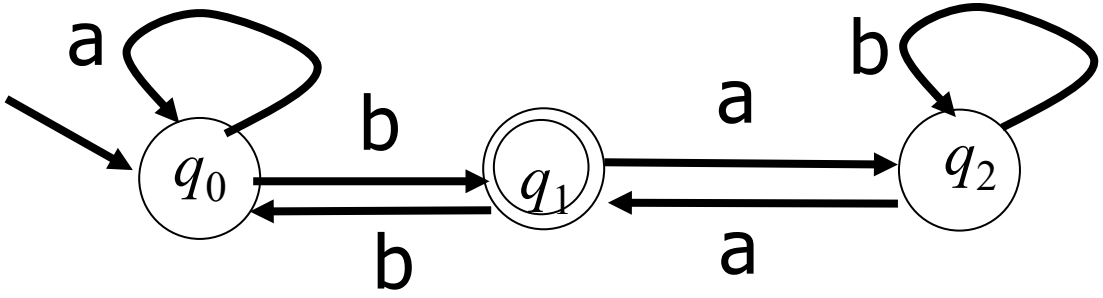
- Step 0. q_iからq_jへ直接遷移するか，または，q_0だけを通過して遷移する記号列からなる集合を表す正則表現R^0_{ij}を求める



$$R^0_{11} = R_{11} + R_{10}(R_{00})^* R_{01}$$
$$= (\varepsilon) + (\mathbf{b} + \varepsilon)(\mathbf{a} + \varepsilon)^* (\mathbf{b} + \varepsilon)$$
$$= \varepsilon + \mathbf{b}\,\mathbf{a}^*\,\mathbf{b}$$

McNorton-Yamadaの方法(3)

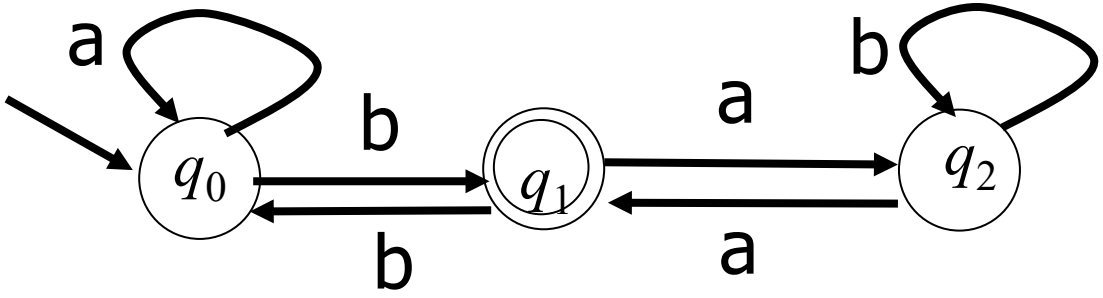
- Step 0. q_iからq_jへ直接遷移するか，または，q_0だけを通過して遷移する記号列からなる集合を表す正則表現R^0_{ij}を求める



$$R^0_{12} = R_{12} + R_{10}(R_{00})^* R_{02}$$
$$= \mathbf{a} + (\mathbf{b} + \varepsilon)(\mathbf{a} + \varepsilon)^* \varnothing$$
$$= \mathbf{a}$$

McNorton-Yamadaの方法(4)

- Step 1. q_iからq_jへ直接遷移するか，または，q_0とq_1を通過して遷移する記号列からなる集合を表す正則表現R^k_{ij}を求める



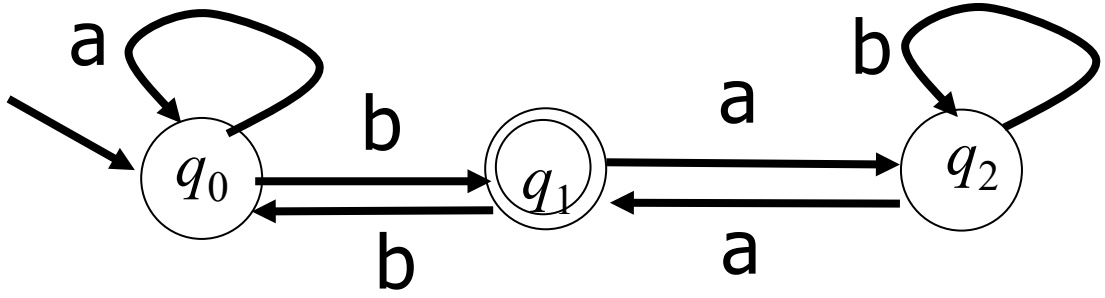
$$R^1_{02} = R^0_{02} + R^0_{01}(R^0_{11})^* R^0_{12} =$$

$$= \varnothing + (\mathbf{a}^*\ \mathbf{b})\ (\varepsilon + \mathbf{b}\ \mathbf{a}^*\ \mathbf{b})^*\ (\mathbf{a})$$

$$= \mathbf{b} + \mathbf{a}^*\ \mathbf{ba} + \varepsilon + \mathbf{a}^*\ \mathbf{b}\ (\mathbf{b}\ \mathbf{a}^*\ \mathbf{b})^*\ \mathbf{a}$$

McNorton-Yamadaの方法(5)

- Step 2. q_iからq_jへ直接遷移するか，または，q_0, q_1, q_2を通過して遷移する記号列からなる集合を表す正則表現R^2_{ij}を求める



$$R^2_{02} = R^1_{02} + R^1_{01}(R^1_{11})^* R^1_{12}$$

McNorton-Yamadaの方法

有限状態オートマトンの状態数をNとする

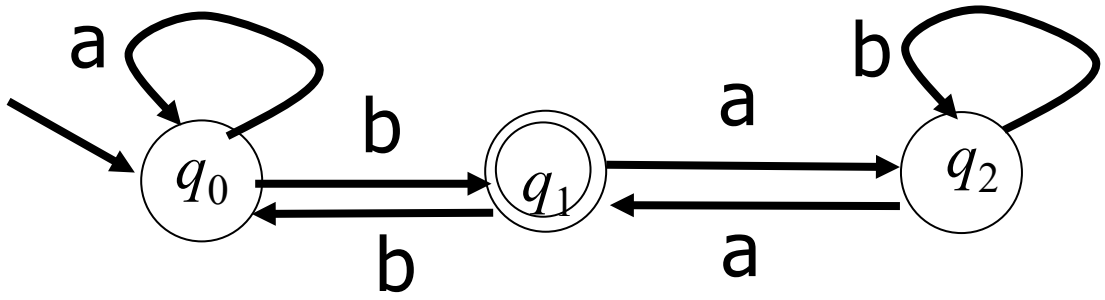
- Step -1. q_iからq_jへ直接遷移する記号だけからなる正則表現$\boldsymbol{R}_{ij}$を求める. ただし, i=jについてはεも加える

k =0,…, N-1について順に

- Step k. q_iからq_jへ直接遷移するか, または, q_0,…,q_kを通過して遷移する記号列の集合を表す正則表現$\boldsymbol{R}^k_{ij}$を求める

有限オートマトンから正則表現へ

- Step 0. q_iからq_jへ直接遷移するか，または，q_0だけを通過して遷移する記号列を正則表現にする



$$R^0_{01} = R_{01} + R_{00}(R_{00})^* R_{10} =$$

$$R^0_{11} = R_{11} + R_{10}(R_{00})^* R_{01} =$$

$$R^0_{00} = R_{00} + R_{00}(R_{00})^* R_{00}$$

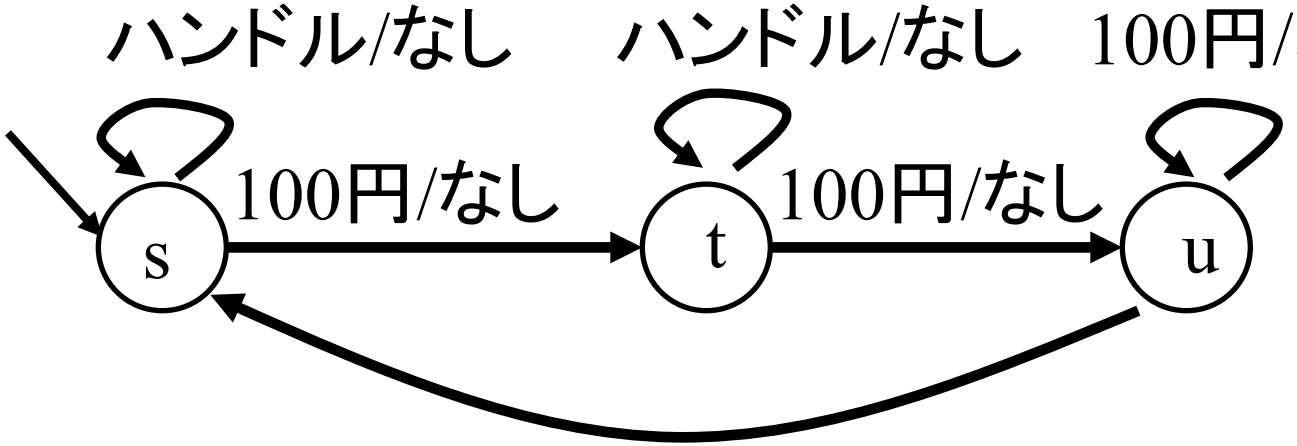
$$= (\mathbf{a} + \varepsilon) + (\mathbf{a} + \varepsilon)(\mathbf{a} + \varepsilon)^* (\mathbf{a} + \varepsilon)$$



順序機械

Mealy型順序機械

- 100円硬貨専用で200円商品の自動販売機



ハンドル/商品

	100円	ハンドル
s	t/なし	s/なし
t	u/なし	t/なし
u	u/なし	s/商品



Mealy型順序機械の表現

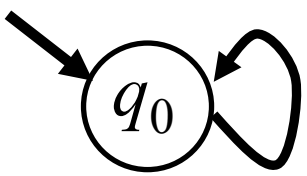
- 以下のような構成要素からなる組$M = (\Sigma, \Delta, S, \delta, \lambda, q_0)$をMealy型順序機械とよぶ.
  - Σ は入力アルファベット
  - Δ は出力アルファベット
  - S は空でない集合であり，その要素を状態とよぶ.
  - δ は $S \times \Sigma \rightarrow S$ なる関数
  - λは $S \times \Sigma \rightarrow \Delta$なる関数
    - δと λは,組$(\delta(q_i, a_j), \lambda(q_i, a_j)) = (q_{ij}, c_{ij})$を用いて上のような表で表現される
  - $q_0 \in S$ は特定の状態であり，初期状態とよぶ
    - 順序回路などでは初期状態を指定しない

	a_1	…	a_n
q_0	(q_{0_1}, c_{0_1})		(q_{0_n}, c_{0_n})
…			
q_m	(q_{m_1}, c_{m_1})		(q_{m_n}, c_{m_n})

Mealy型順序機械の例(1)

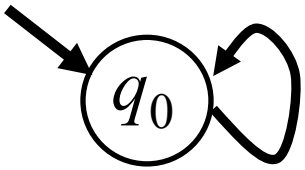
- 記号の変換

$\Sigma = \{0,1\}, \Delta = \{a, b\}$



10010011 ⇒ abbabbaa

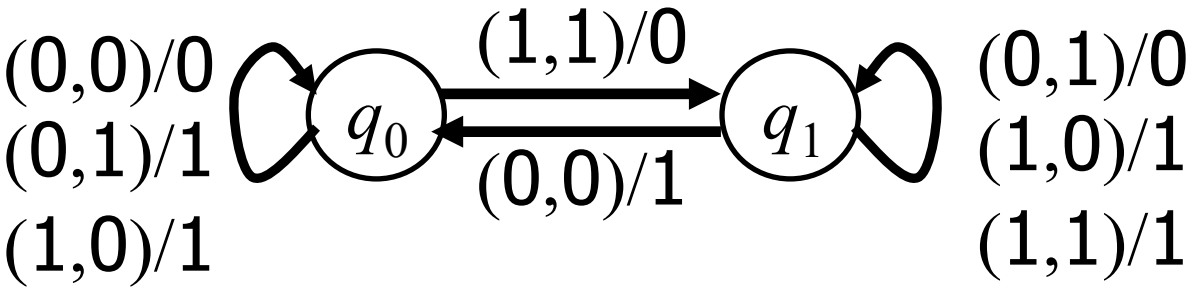
- 記号の変換 $\Sigma = \{0,1,2,3\}, \Delta = \{00, 01, 10, 11\}$



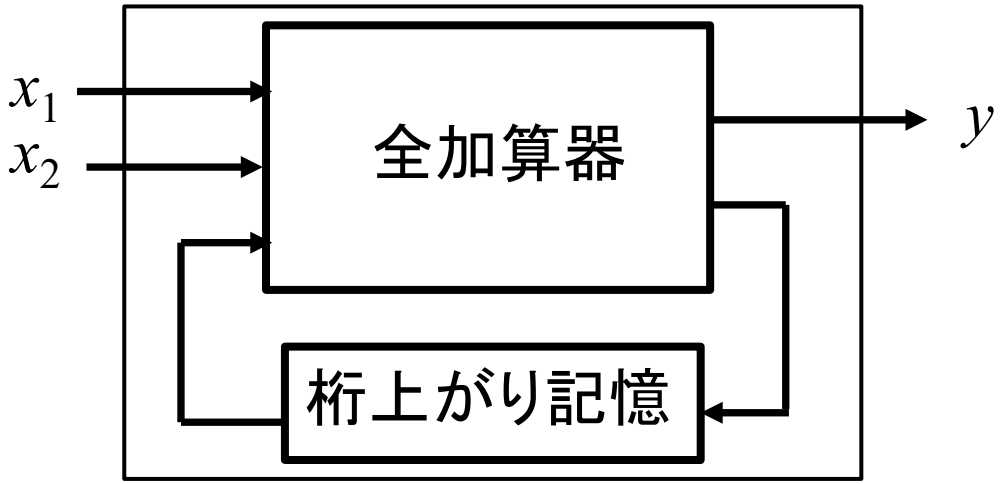
2130 ⇒ 10011100

Mealy型順序機械の例(2)

- 直列加算器　$\Sigma = \{(0,0), (0,1), (1,0), (1,1)\}, \Delta = \{0,1\}$

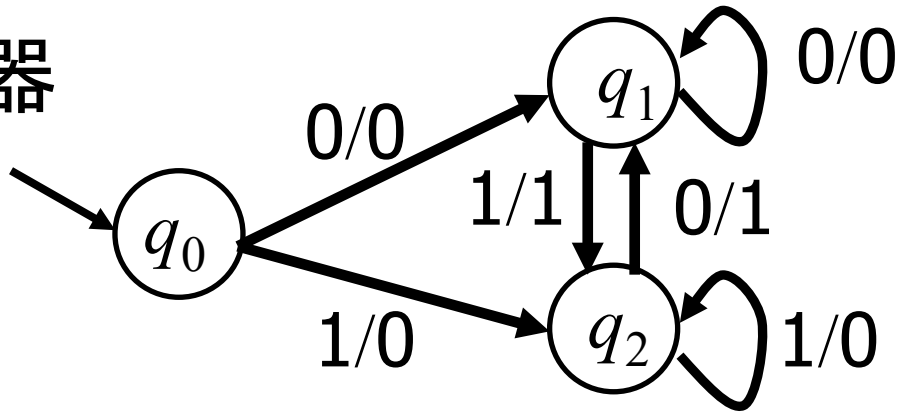


- 入力 x_1, x_2には2つの自然数の2進表記が下位の桁から順に送られる



Mealy型順序機械の例(3)

エッジ検出器



0000000000	⟹	0000000000
0001111100	⟹	0001000010
0001111100	⟹	0001000010
0000111000	⟹	0000100100
0000111000	⟹	0000100100
0000010000	⟹	0000011000
0000010000	⟹	0000011000
0000000000	⟹	0000000000

Mealy型順序機械の例(4)







[Wikipediaより]　[https://jack8.at.webry.info/201203/article_3.html]

1-800-MY-APPLE ⇨ 1-800-69-17753 米国アップル社 コールセンター

1-800-JAL-FONE ⇨ 1-800-525-3663 米国JAL コールセンター

Moore型順序機械

- 以下のような構成要素からなる組$M=(\Sigma,\Delta, S, \delta, \lambda, q_0)$をMoore型順序機械とよぶ.
  - Σ は入力アルファベット
  - Δ は出力アルファベット
  - S は空でない集合であり，その要素を状態とよぶ.
  - δ は $S\times\Sigma \rightarrow S$ なる関数
  - λは $S \rightarrow \Delta$なる関数
    - δと λは右のような表で表現される
  - $q_0 \in S$ は特定の状態であり，初期状態とよぶ
    - 順序回路などでは初期状態を指定しない
    - $\Delta=\{0, 1\}$のとき 言語を受理する有限状態オートマトンとみなすことができる

	a_1	…	a_n	λ
q_0	q_{0_1}		q_{0_n}	c_0
…				
q_m	q_{m_1}		q_{m_n}	c_m

Moore型順序機械の例(1)

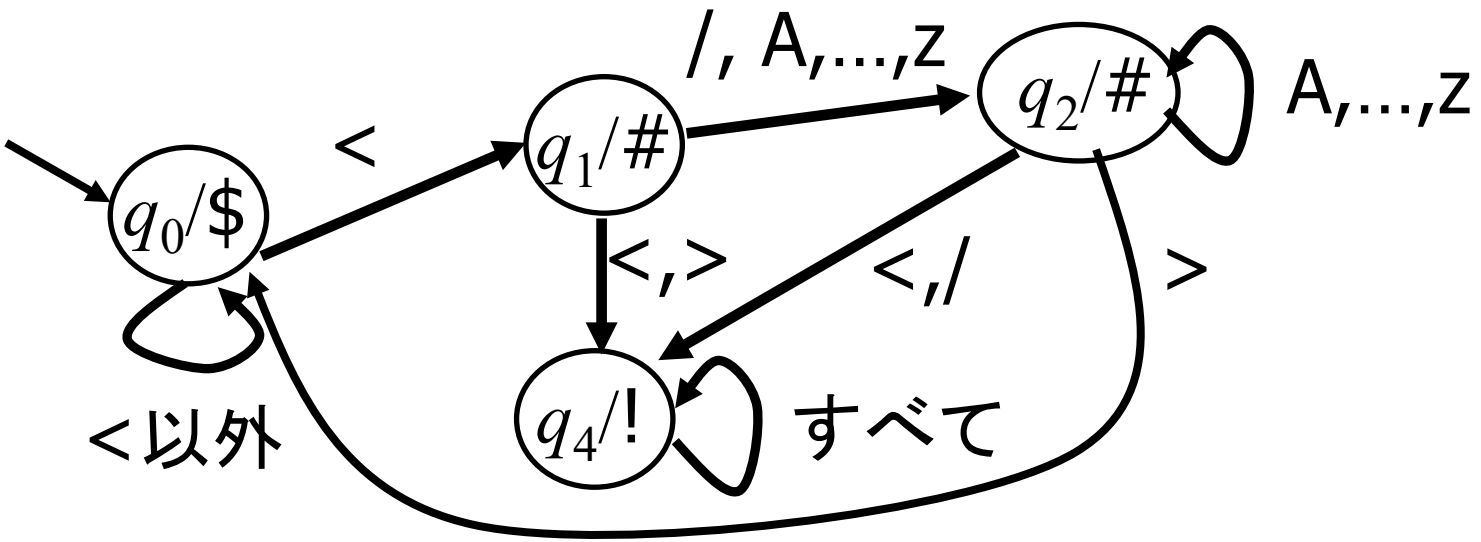
- 字句(トークン)解析 $\Sigma = \{<, >, /, A,..., z\}$,

  $\Delta = \{\#, \$, !\}$

  - プログラミング言語などでは，処理の最小単位をトークンとよぶ

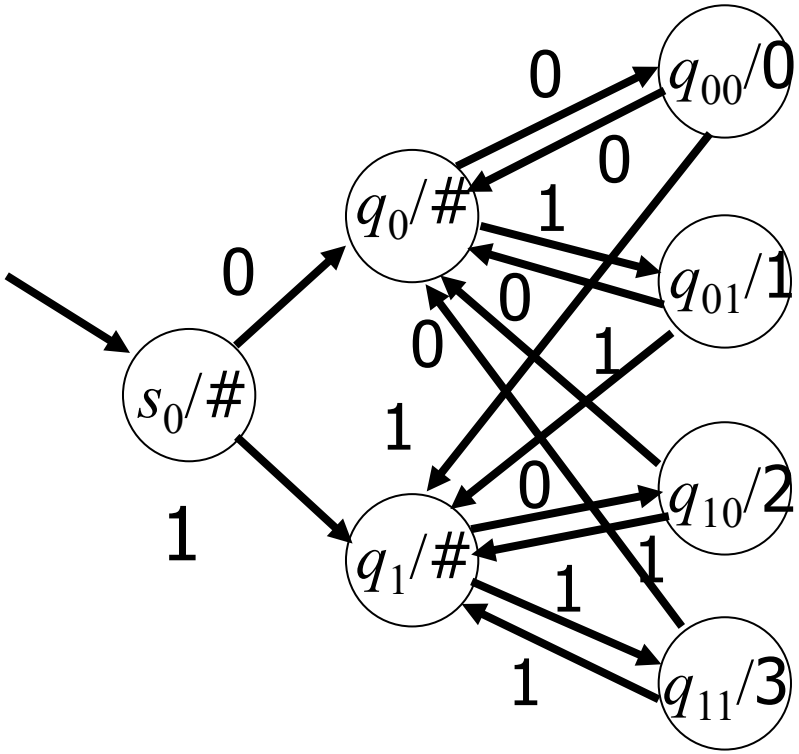
例 HTMLのタグは‘<’ と ‘>’で挟まれた記号列

または ‘</’ と ‘>’で挟まれた記号列または‘</>’



Moore型順序機械の例(2)

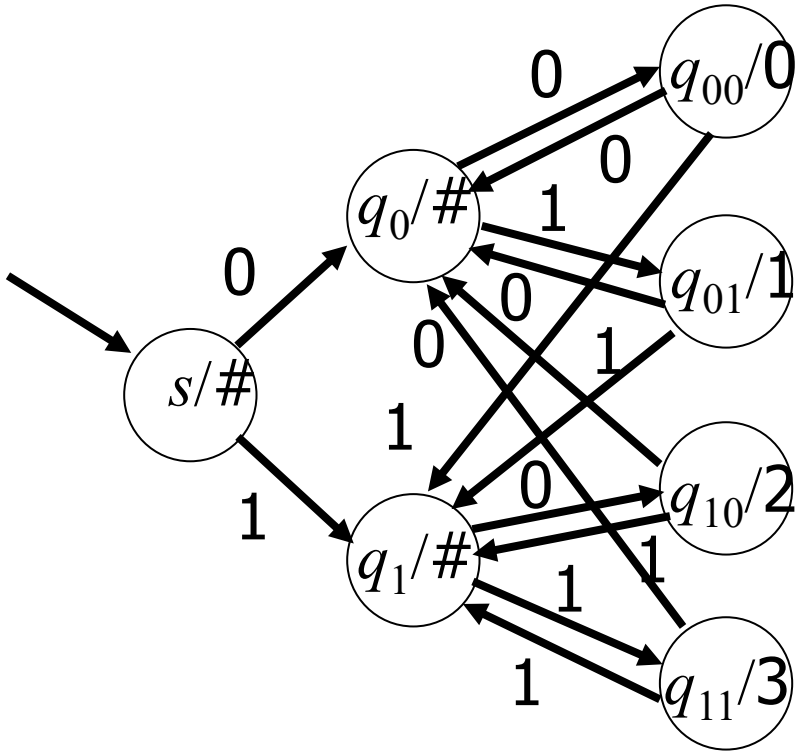
- 2進表現から4進表現への変換

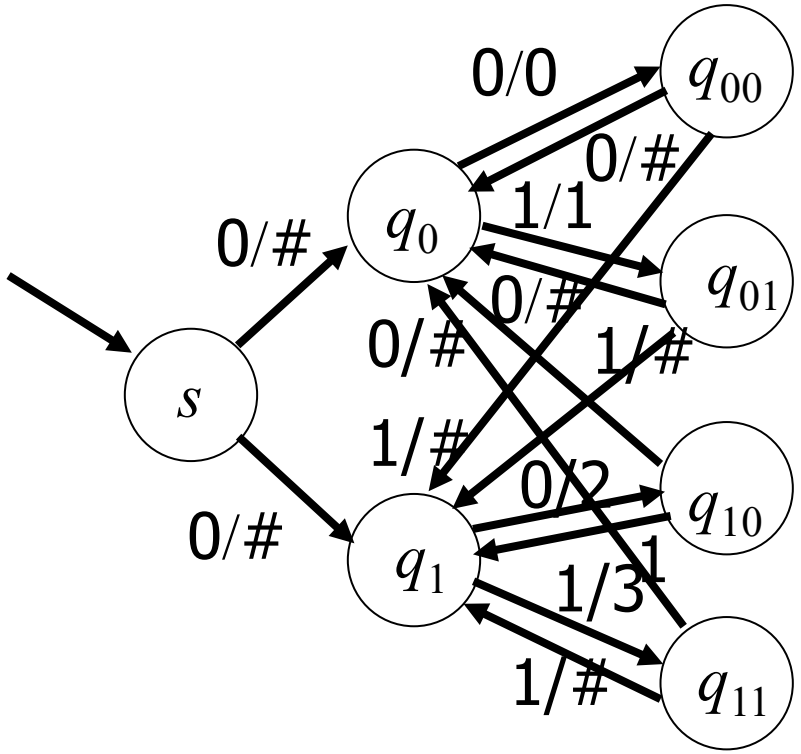


10010011 → ##2#1#0#3

例

- 2進表現から4進表現への変換





10010011 ⇒ #2#1#0#3

Moore型からMealy型への変換

もとのMoore型を $M = (\Sigma, \Delta, S, \delta, \lambda, q_0)$とし

- 出力のタイミングを前にずらす
- 記号列を読み込む直前の初期状態の出力を取り除く

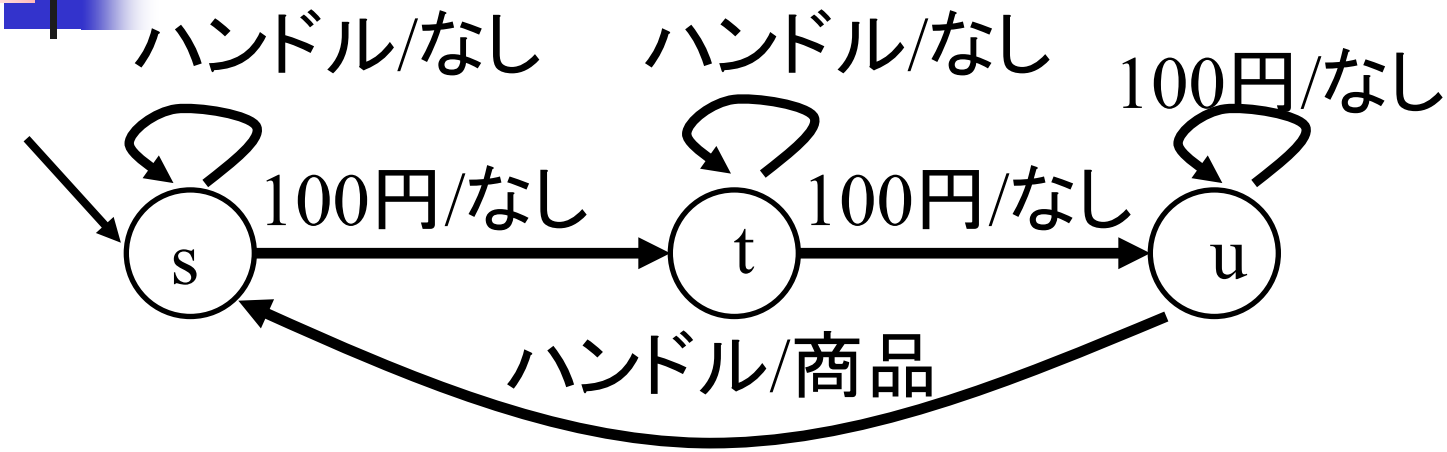
を認めれば,

状態qでの出力$\lambda(q)$を, qに至る遷移において出力する

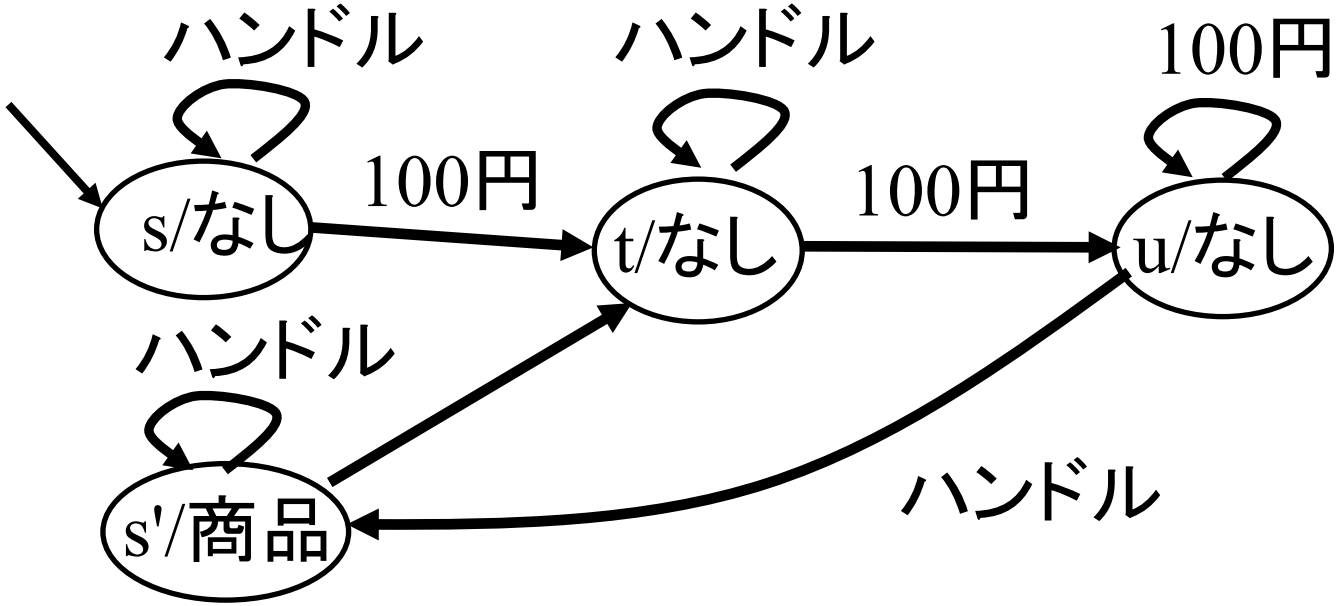
$$\lambda'(q, a) = \lambda(\delta(q, a))$$

とすれば, Mealy型$M' = (\Sigma, \Delta, S, \delta, \lambda', q_0)$ に変換できる

Mealy型順序機械とMoore型順序機械







Mealy型からMoore型への変換

もとのMealy型を $M = (\Sigma, \Delta, S, \delta, \lambda, q_0)$とし, 求めるMoore型を$M' = (\Sigma, \Delta, S', \delta', \lambda', q_0)$ とする

- 出力のタイミングを後ろにずらす
- 記号列を読み込む直前の初期状態の出力を定義しないを認める.
- Mの状態と出力記号の組を新たに状態と考える

$$S' = S \times \Delta$$

Mが状態qに至る遷移においてcを出力するとすれば,
M'においては(q,c)に遷移して cを出力する, とする

$$\delta'((q, c), a) = (\delta(q, a), \lambda(\delta(q, a))$$
$$\lambda'((q, c)) = c$$



オートマトンによるキーワード探索

キーワード探索とは?

- テキスト探索, パターン検出, パターン照合, 記号列照合などともよばれる

入力: (長い)記号列 s, (短い)キーワード w

出力: 記号列 s で w が出現したすべての位置

例 s = asukaratokyotokyotoniiku

w = tokyoto

asukaratokyotokyotoniiku

00000000000010000100000

前綴り(prefix)と後綴り(suffix)

キーワード $w = c_1 c_2 c_3 \cdots c_k$

- (キーワードの)前綴り $c_1 c_2 c_3 \cdots c_j \quad j = 1, \ldots, k$

  tokyoto の前綴り t, to, tok, toky, …, tokyoto
- (キーワードの)後綴り $c_j c_{j+1} c_{j+2} \cdots c_k \quad j = 1, \ldots, k$

  tokyoto の前綴り tokyoto, okyoto, …, oto, to, o

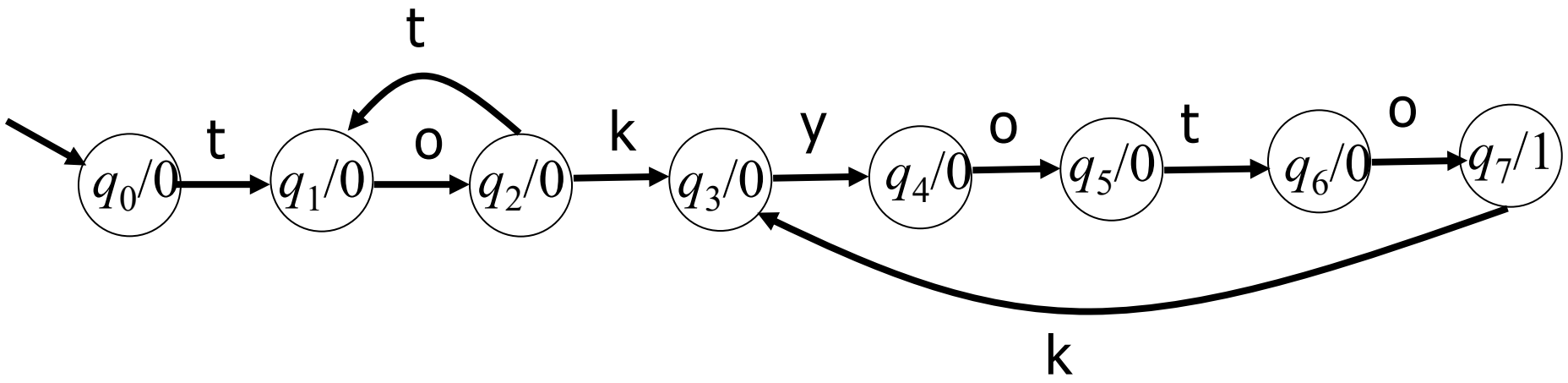
前綴り(prefix)から順序機械へ

キーワード w = tokyoto

入力記号cを加えたときの前綴りの前綴りと後綴りに注目

tokt

tokyotok



前綴り(prefix)と後綴り(suffix)

キーワード $w = c_1 c_2 c_3 \cdots c_k$

- (キーワードの)前綴り $c_1 c_2 c_3 \cdots c_j \quad j=1,\ldots,k$

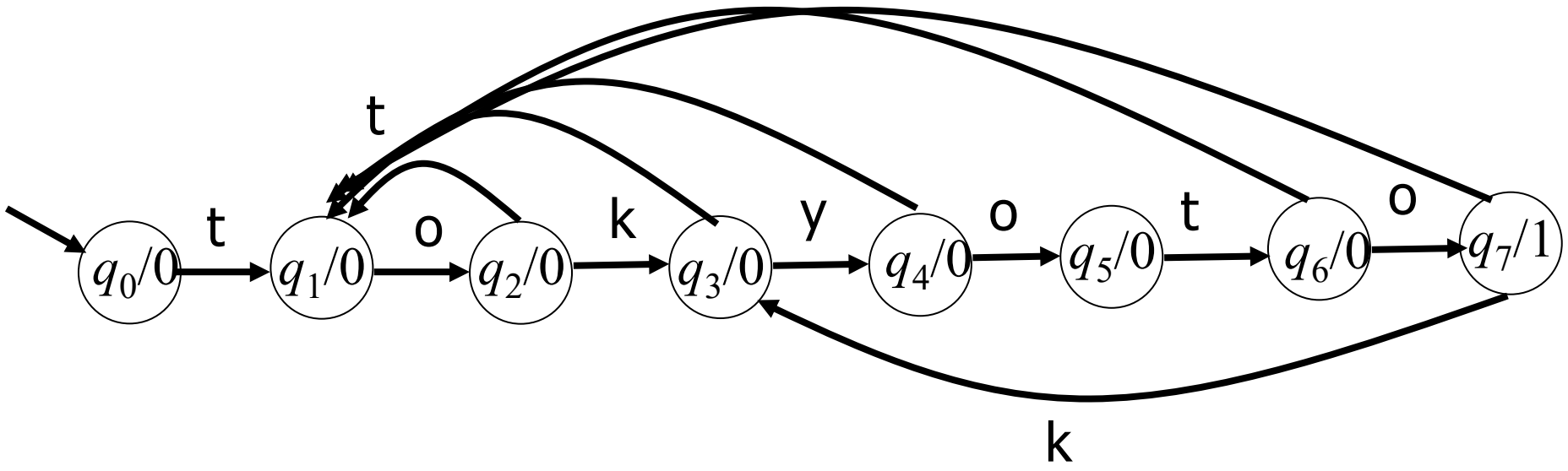
  tokyoto の前綴り t, to, tok, toky, …, tokyoto
- (キーワードの)後綴り $c_j c_{j+1} c_{j+2} \ldots c_k \quad j=1,\ldots,k$

  tokyoto の前綴り tokyoto, okyoto, …, oto, to, o

前綴り(prefix)から順序機械へ(1)

キーワード $w =$ tokyoto

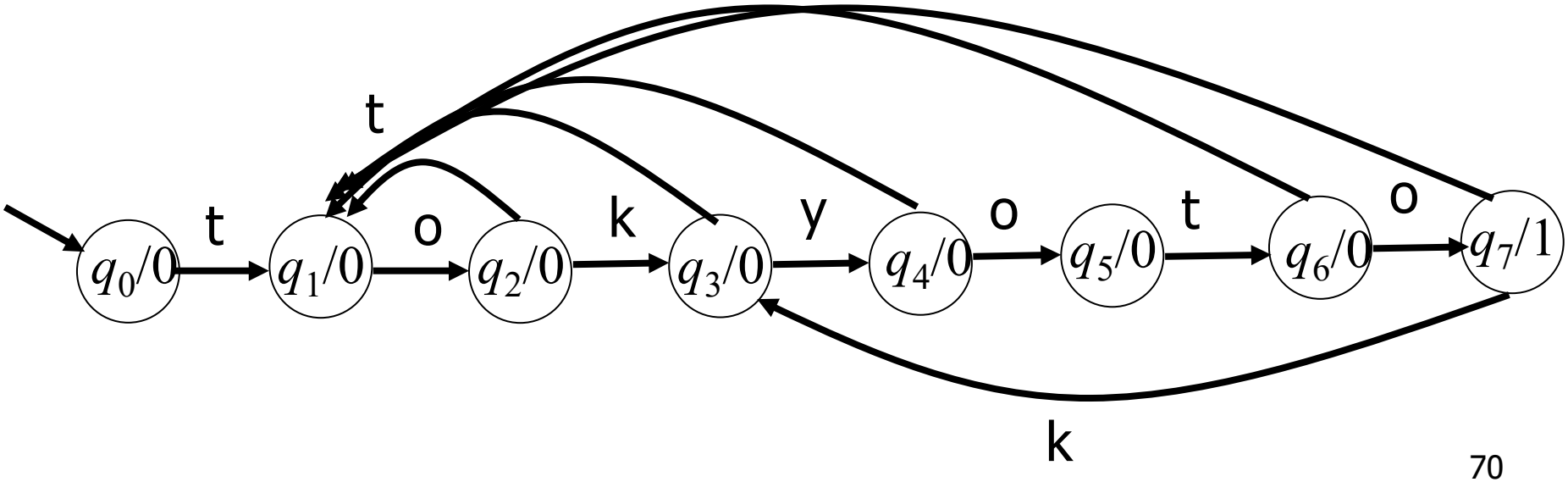
入力記号cを加えたときの前綴りの前綴りと後綴りに注目



前綴り(prefix)から順序機械へ(2)

キーワード $w =$ tokyoto

- 入力記号cを加えたときの前綴りの前綴りと後綴りに注目
- 図示した遷移以外はすべてq_0に遷移する



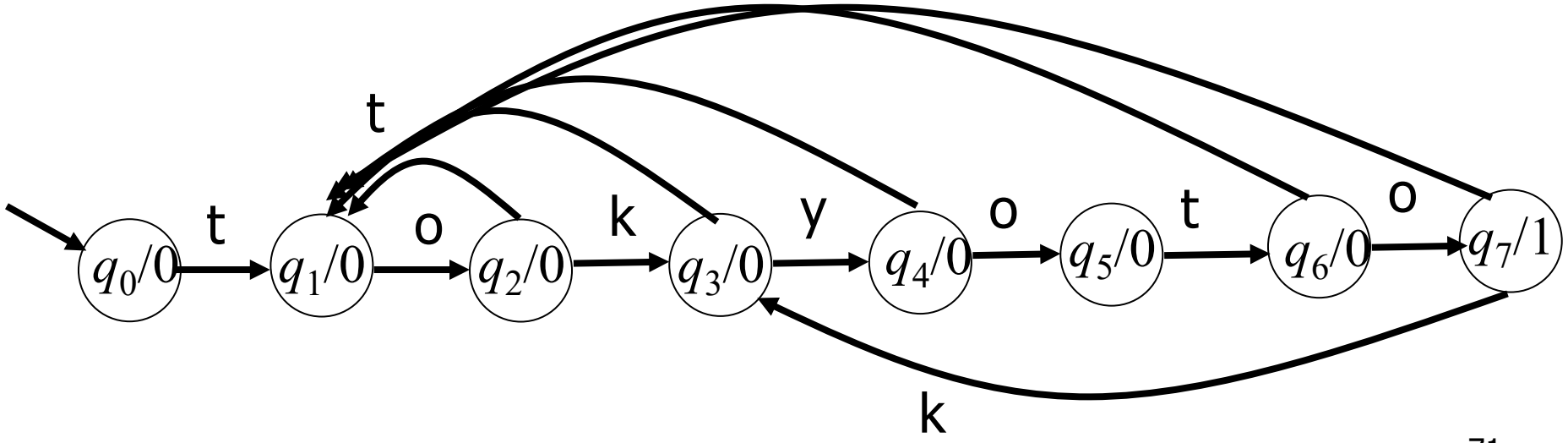
前綴り(prefix)から順序機械へ(3)

キーワード w = tokyoto

- 入力記号cを加えたときの前綴りの前綴りと後綴りに注目

  tokt

  tokyotok



前綴り(prefix)から順序機械へ(4)

キーワード w = tokyoto

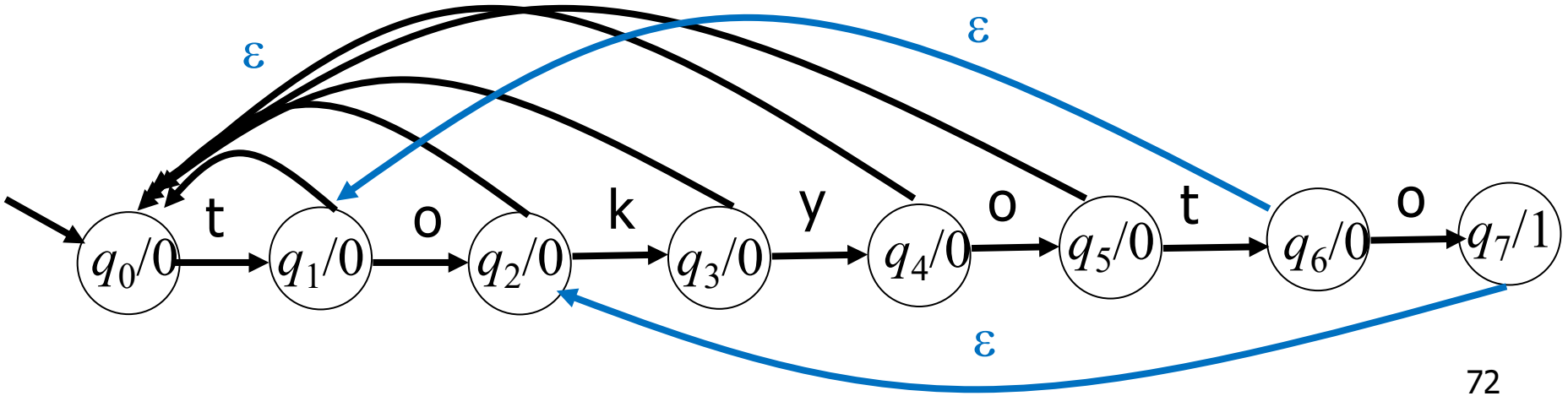
- 入力記号cを加えたときの前綴りの前綴りと後綴りに注目

  tokt

  tokyotok

- 失敗関数を導入

  遷移できない場合に限定したε遷移(失敗遷移)



順序機械としての定式化

- Aho-Corasickオートマトン$M=(\Sigma,\Delta, S, \delta, h, \lambda, q_0)$は次の要素からなる.
  - Σ は入力アルファベット
  - $\Delta=\{0, 1\}$
  - S は状態の集合
  - δ は $S\times\Sigma \to S\cup\{\mathrm{fail}\}$なる関数 かつ

    $\delta(q_0, c) \neq \mathrm{fail}\ \ (c \in \Sigma)$
  - h は $S \to S$ なる関数 かつ $\mathrm{depth}(h(q)) < \mathrm{depth}(q)$
    - 失敗関数とよばれる
    - $\mathrm{depth}(q)$はq_0からqに至る最短経路長
  - λは $S \to \Delta =\{0, 1\}$なる関数
    - δと λは右のような表で表現される
  - $q_0 \in S$ は初期状態

Aho-Corasickオートマトンの動作

入力: 記号列 $a_1 a_2 a_3 \ldots a_n$

1. $q = q_0$
2. $i = 1$
3. $\delta(q, a_i) \neq \text{fail}$ ならば

   $q := \delta(q, a_i)$, $\lambda(q)$を出力し, i を1増やす

   $\delta(q, a_i) = \text{fail}$ ならば $q := h(q)$
4. $i > n$ ならば終了

   $i \leq n$ ならば3へ戻る

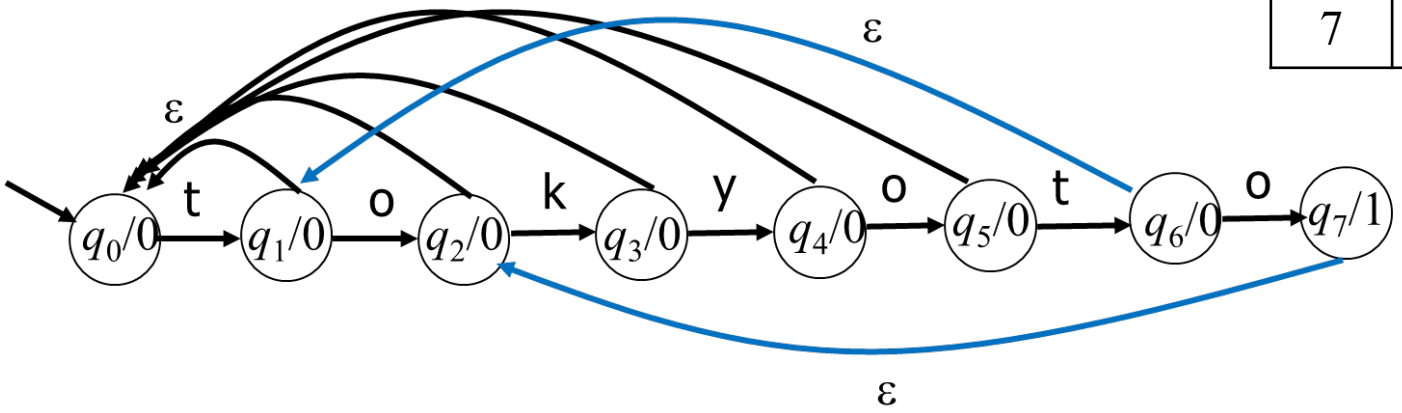
失敗関数の構成(例1)

キーワード w = tokyoto

- 各状態q_uまでの語u(wの前綴り)について，
  wの前綴りになっていて，かつ
  uの後つづりになっている

ような最長のvを見つける.

n	u	$f(n)$
0		0
1	t	0
2	to	0
3	tok	0
4	toky	0
5	tokyo	0
6	tokyot	1
7	tokyoto	2



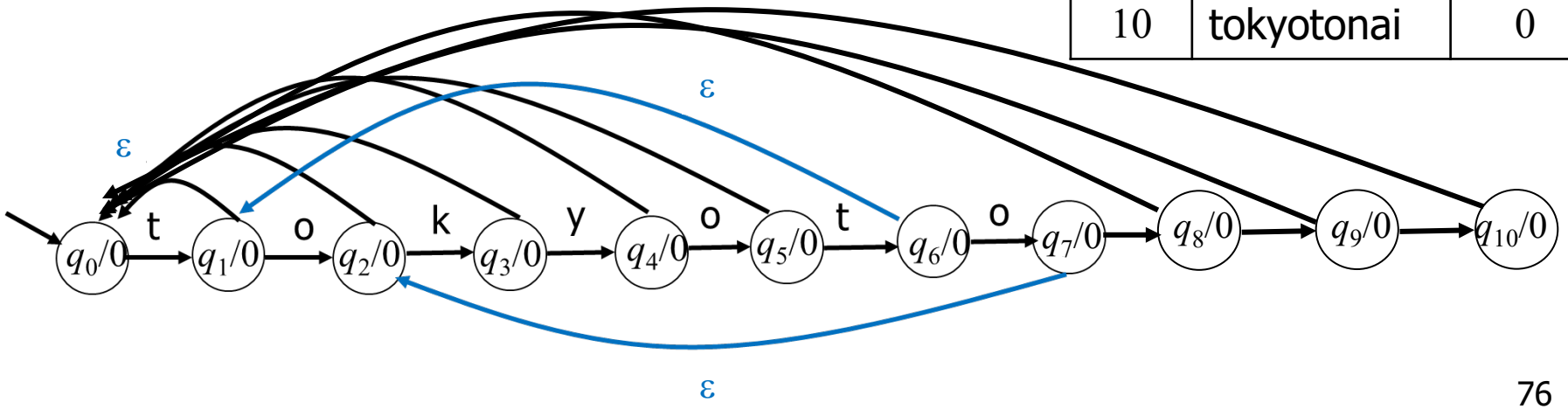
失敗関数の構成(例2)

キーワード w = tokyotonai

- 各状態q_uまでの語u(wの前綴り)について，
  wの前綴りになっていて，かつ
  uの後つづりになっている

ような最長のvを見つける.

n	u	$f(n)$
0		0
1	t	0
2	to	0
3	tok	0
4	toky	0
5	tokyo	0
6	tokyot	1
7	tokyoto	2
8	tokyoton	0
9	tokyotona	0
10	tokyotonai	0



失敗関数の構成手順

1. $f(q_0):=q_0$
2. $f(q_1):=q_0$
3. $i = 1$から順に4を繰り返す
4. $\delta(q_i, a)=q_{i+1}$ならば$f(q_{i+1}):= \delta(r, a)$

ただし, 状態 r は以下の手順で決める

$r:=f(q_i)$

while $\delta(r, a)$==fail **do**

$r:=f(r)$

n	u	$f(n)$
0		0
1	t	0
2	to	0
3	tok	0
4	toky	0
5	tokyo	0
6	tokyot	1
7	tokyoto	2
8	tokyoton	0
9	tokyotona	0
10	tokyotonai	0

失敗関数の構成(例3)

キーワード w = abaababa

- 各状態q_uまでの語u(wの前綴り)について，
  wの前綴りになっていて，かつ
  uの後つづりになっている

ような最長のvを見つける.

(注) 以下の漸化式で定義される語 (文字列) F_n をFibonatti文字列という.

$F_0 = \varepsilon$

$F_1 = \mathrm{b}$

$F_2 = \mathrm{a}$

$F_n = F_{n-1}\,F_{n-2}\ (n \geq 2)$

n	u	$f(n)$
0		0
1	a	0
2	ab	0
3	aba	1
4	abaa	1
5	abaab	2
6	abaaba	3
7	abaabab	2
8	abaababa	3

不要な失敗遷移の除去(例-1)

入力:tokyotawa

キーワード w = tokyoto

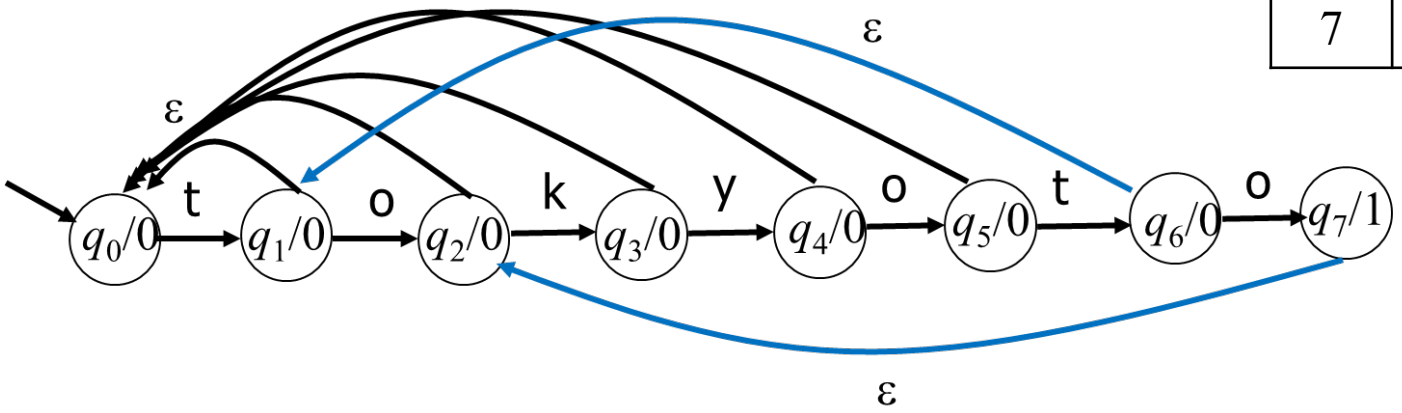
- tokyot まで読込むと状態は q_6

  次の入力は a である

  $f(q_6)=q_1$ であるが q_1からの遷移は失敗

  $f(q_1)=q_0$

n	u	$f(n)$
0		0
1	t	0
2	to	0
3	tok	0
4	toky	0
5	tokyo	0
6	tokyot	1
7	tokyoto	2



不要な失敗遷移の除去(例1-2)

入力:tokyotawa

キーワード w = tokyoto

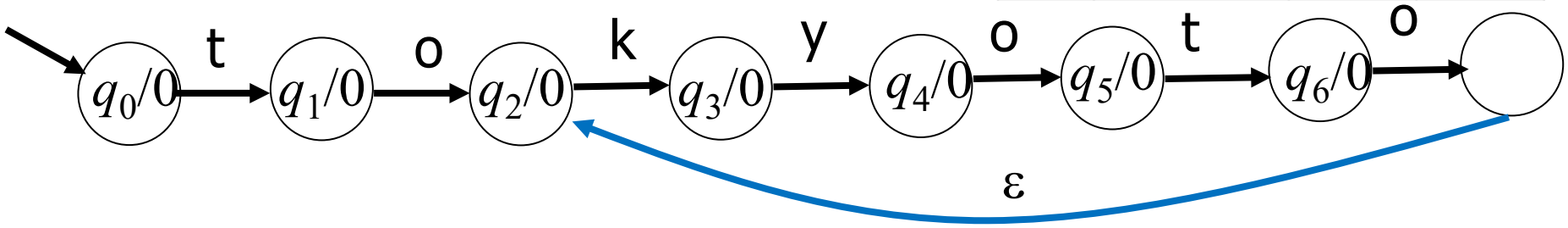
- tokyot まで読込むと状態は q_6

  次の入力は a である

  $f(q_6)=q_1$ であるが q_1からの遷移は失敗

  $f(q_1)=q_0$

n	u	$f(n)$	$h(n)$
0		0	0
1	t	0	0
2	to	0	0
3	tok	0	0
4	toky	0	0
5	tokyo	0	0
6	tokyot	1	0
7	tokyoto	2	2

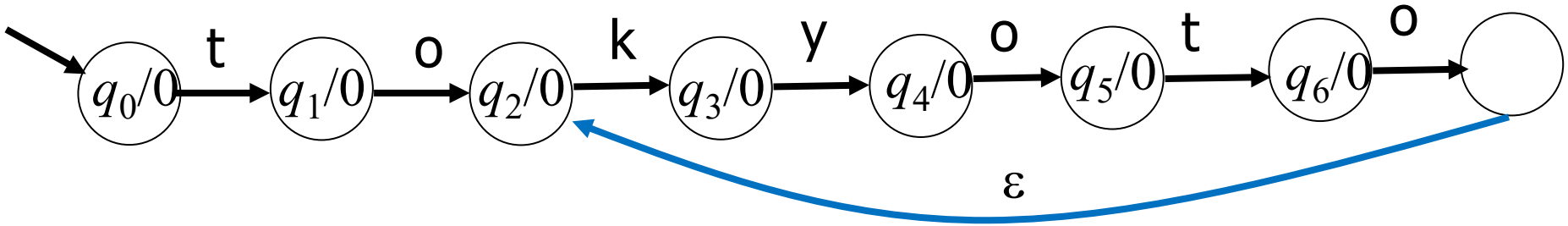


不要な失敗遷移の除去(例1)

tokt

tokyotok

n	u	$f(n)$	$h(n)$
0		0	0
1	t	0	0
2	to	0	0
3	tok	0	0
4	toky	0	0
5	tokyo	0	0
6	tokyot	1	0
7	tokyoto	2	2



不要な失敗遷移の除去(例3)

- キーワード abaababa

n	u	$f(n)$	$h(n)$
0		0	0
1	a	0	0
2	ab	0	0
3	aba	1	1
4	abaa	1	0
5	abaab	2	0
6	abaaba	3	3
7	abaabab	2	0
8	abaababa	3	3

不要な失敗遷移の除去手順

1. $f(q_0) := q_0$
2. $i = 0$ から順に3を繰り返す
3. $\delta(f(q_i), a) \neq \text{fail}$ であるようなaに対して
   もし $\delta(q_i, a) \neq \text{fail}$ であるならば
   $h(q_i, a) = h(f(q_i), a)$
   そうでなければ
   $h(q_i, a) = f(q_i)$

n	u	$f(n)$	$h(n)$
0		0	0
1	t	0	0
2	to	0	0
3	tok	0	0
4	toky	0	0
5	tokyo	0	0
6	tokyot	1	0
7	tokyoto	2	2

複数個のキーワードの扱い(1)

- キーワードが複数個ある場合に対しても同じ考え方が適用可能

例 s = asukaratokyotokyotoniiku

w_1 = tokyo

w_2 = kyoto

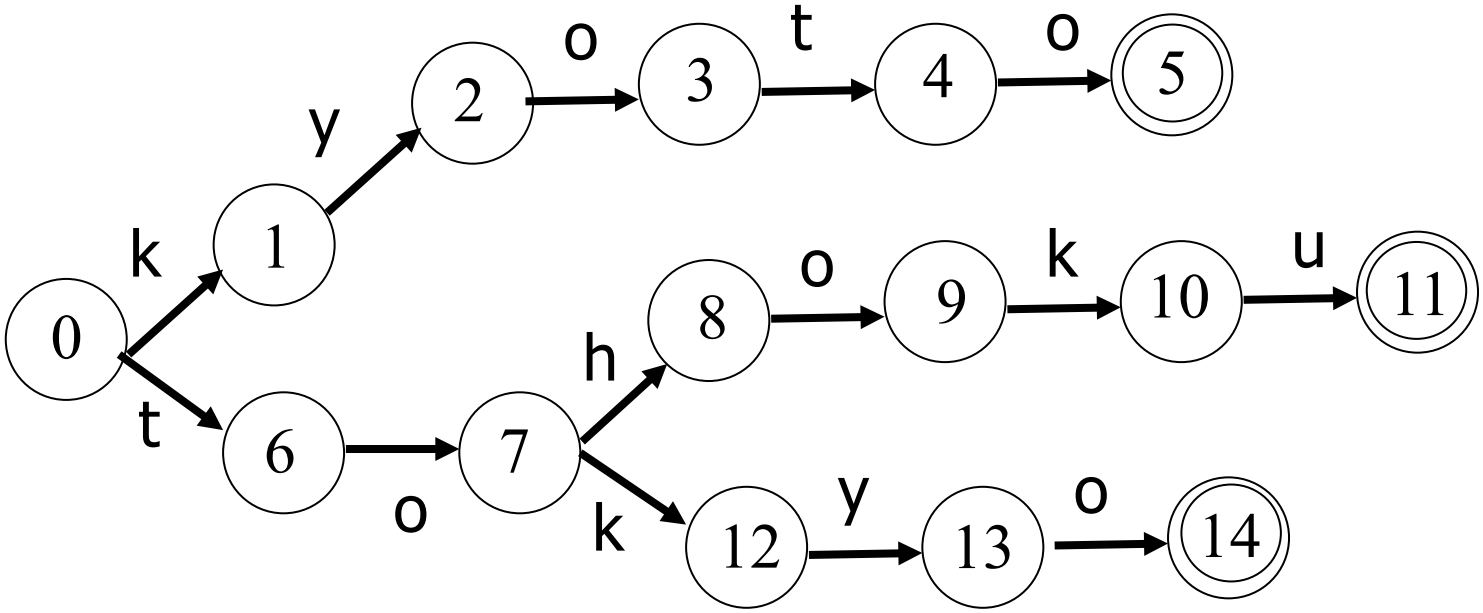
kyotoasudetokyotokyotoniiku

00002000000000102001020000 0

前綴り木(prefix tree, trie)

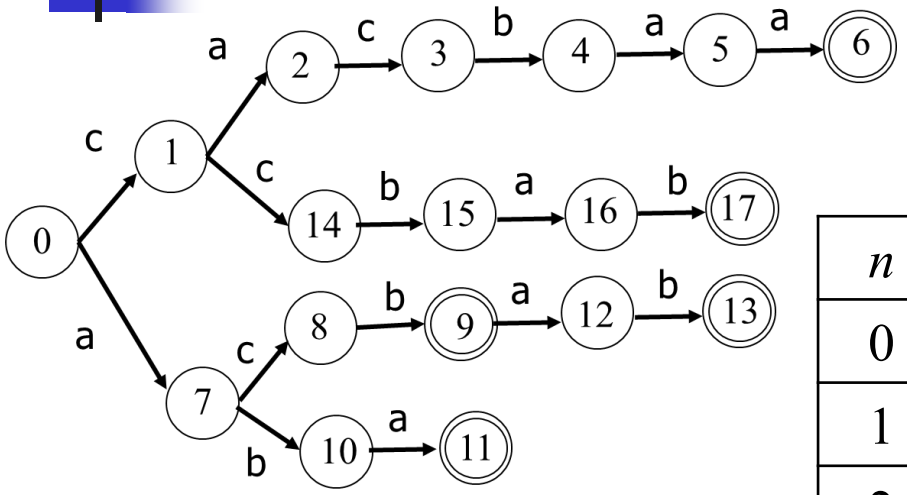
- キーワードの前綴りの重複を表現した木

  tokyo　kyoto　tohoku



- 失敗関数は，構成アルゴリズム中のq_iのiを前綴木の深さに読み替えて，深さ優先探索を行う

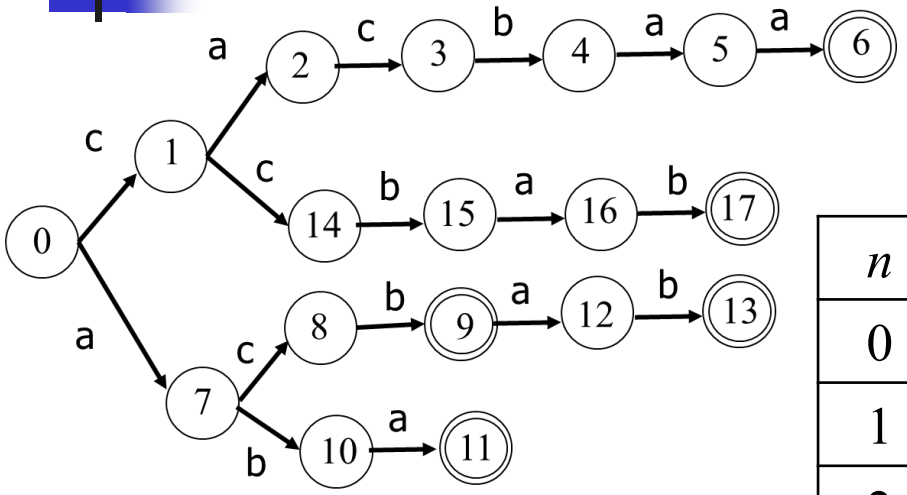
順序機械の構成(例4-2)



n	a	b	c
0	7	0	1
1	2	F	14
2	F	F	3
3	F	4	F
4	5	F	F
5	6	F	F
6	F	F	F
7	F	10	8
8	F	9	F

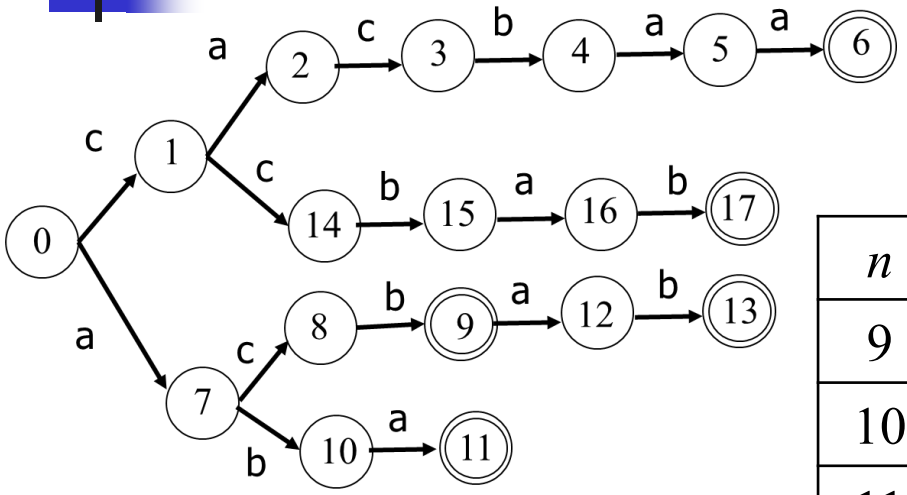
n	a	b	c
9	12	F	F
10	11	F	F
11	F	F	3
12	F	13	F
13	F	F	F
14	F	15	F
15	16	F	F
16	F	17	F
17	F	F	F

順序機械の構成(例4-3)



n	a	b	c	f	h
0	7	0	1	0	0
1	2	F	14	0	0
2	F	F	3	7	7
3	F	4	F	8	1
4	5	F	F	9	0
5	6	F	F	12	12
6	F	F	F	7	7
7	F	10	8	0	0
8	F	9	F	1	1

順序機械の構成(例4-4)



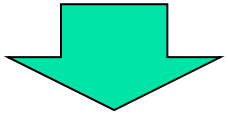
n	a	b	c	f	h
9	12	F	F	0	0
10	11	F	F	0	0
11	F	F	3	7	7
12	F	13	F	7	7
13	F	F	F	10	10
14	F	15	F	1	1
15	16	F	F	0	0
16	F	17	F	7	7
17	F	F	F	10	10

ソフトウェアのモデルとしてのオートマトン

ITシステムの特徴と設計

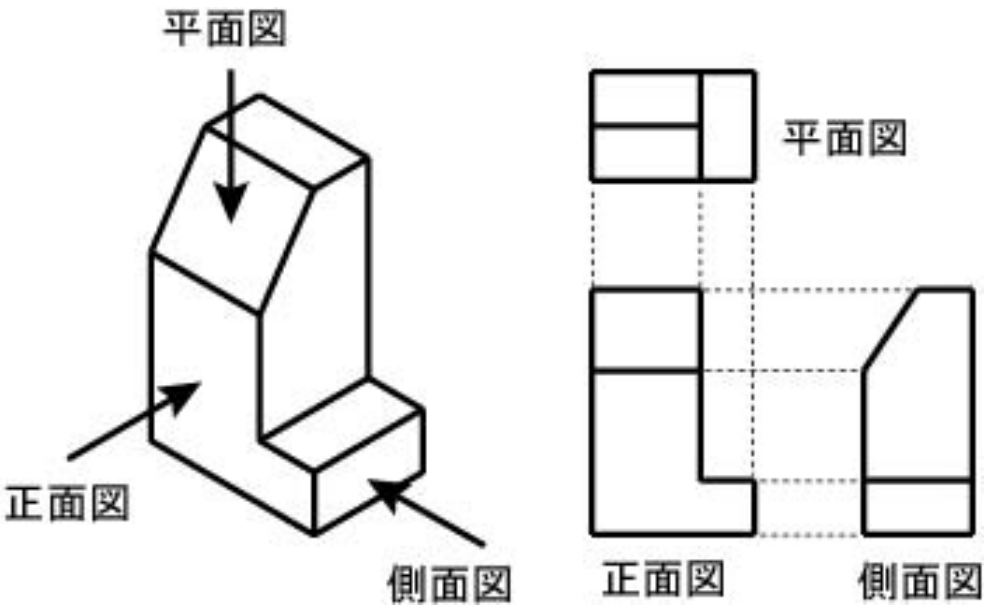
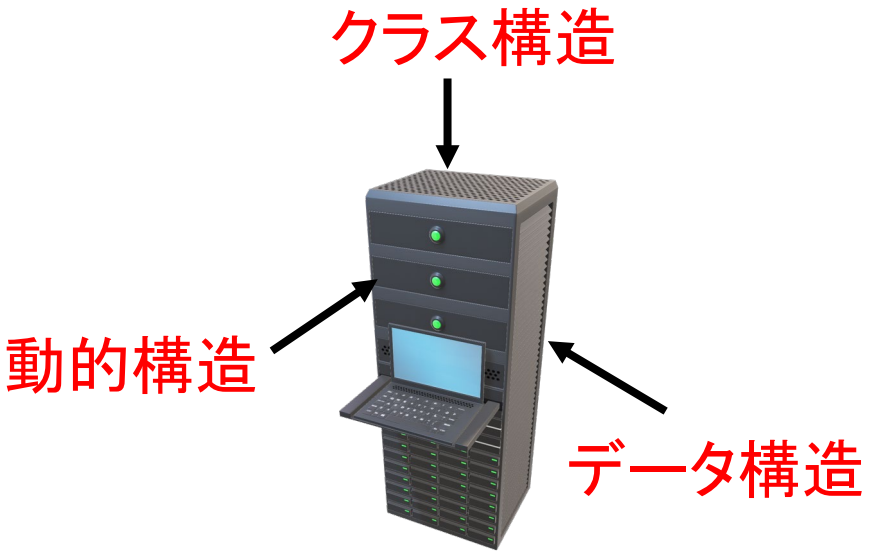
ITシステム=プログラムの集積体

- 動的構造：ITシステムの動作の構造
  - プログラムはITシステムの動作の構造を反映して記述する
  - 代入，条件分岐，繰り返しの3種類で構成される[Dijkstra]
- データ構造：ITシステムの取り扱うデータの構造
  - プログラムはデータの管理の構造を反映して記述する
  - データは構造を入れて初めて管理できる
- プログラム構造
  - 巨大化したプログラムは構造を入れて管理する
  - 編集時の誤りを防ぐ



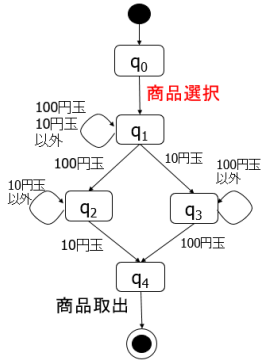
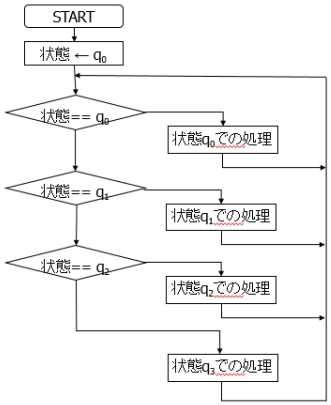
- 3種類の構造を相互に考慮してITシステムを設計する
  - 機械や建築の「三面図」

三面図とITシステム構造の比較

機械・建築などの三面図	
ソフトウェアの構造	

抽象モデル，具体モデル，プログラム

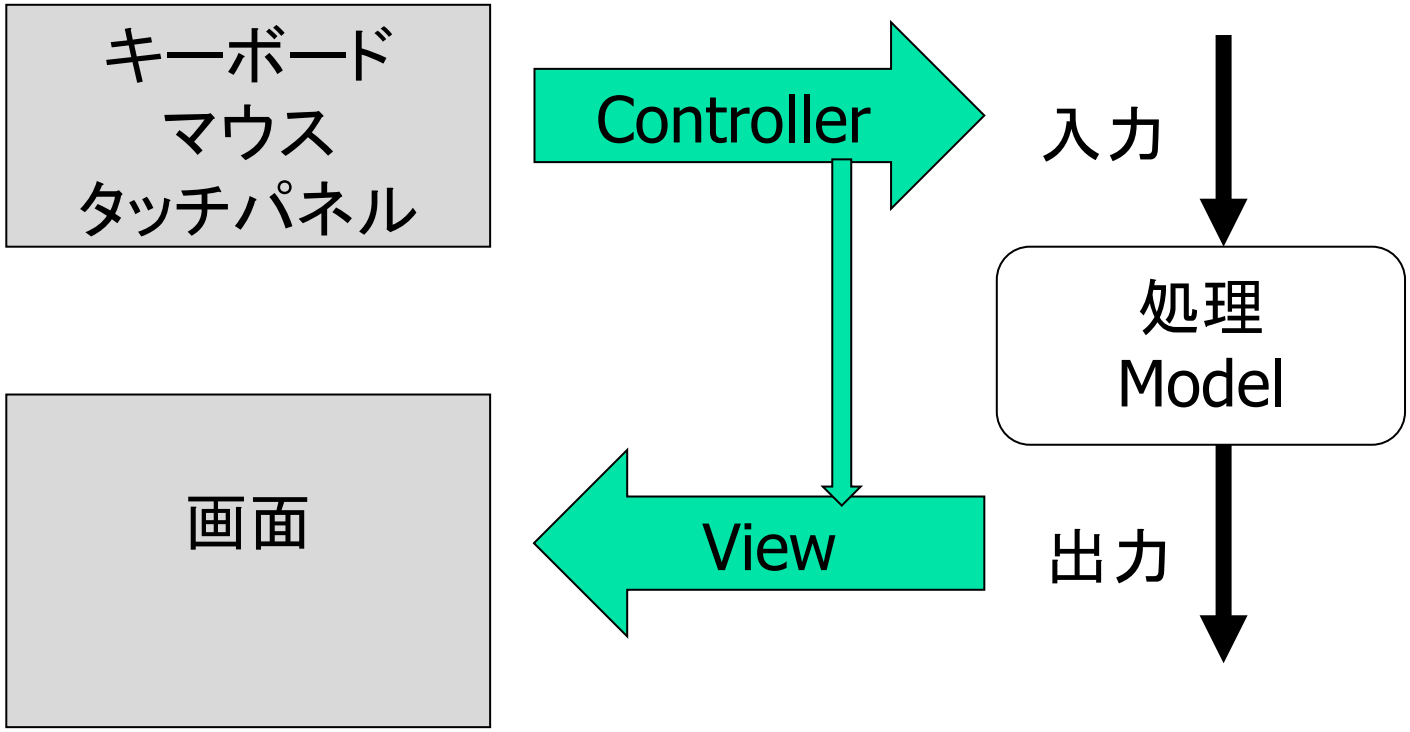
	抽象モデル	具体的モデル	プログラム

動的構造	状態遷移モデル，
	活動系列モデルなど
	 
		フローチャート
		 
			Java, Pythonなど
			`year = int(input("Enter Year: "))`
			
			`# Leap Year Check`
			`if year % 4 == 0 and year % 100 != 0:`
			`  print(year, "is a Leap Year")`
			`elif year % 100 == 0:`
			`  print(year, "is not a Leap Year")`
			`elif year % 400 ==0:`
			`  print(year, "is a Leap Year")`
			`else:`
			`  print(year, "is not a Leap Year")`

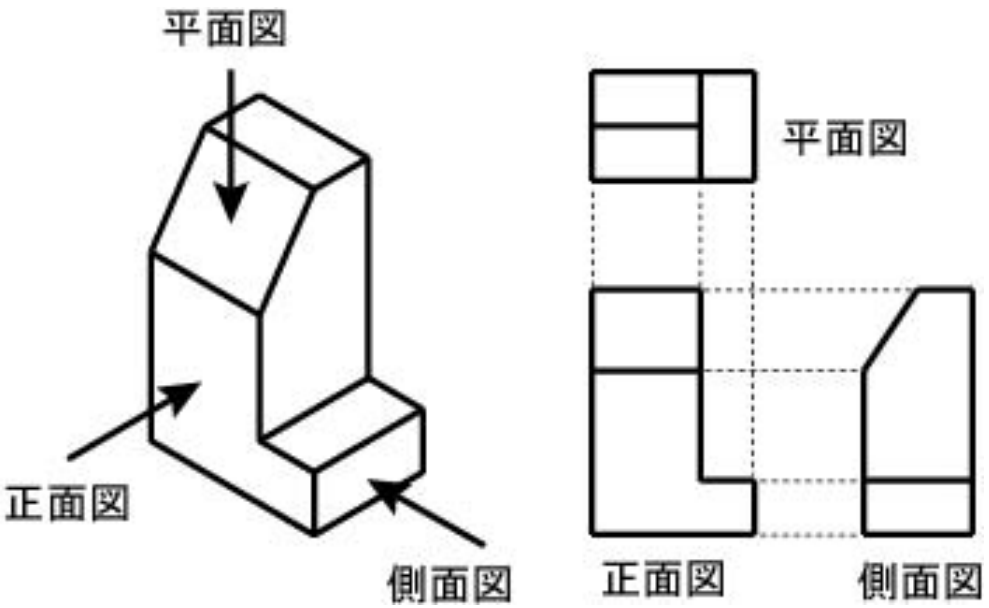
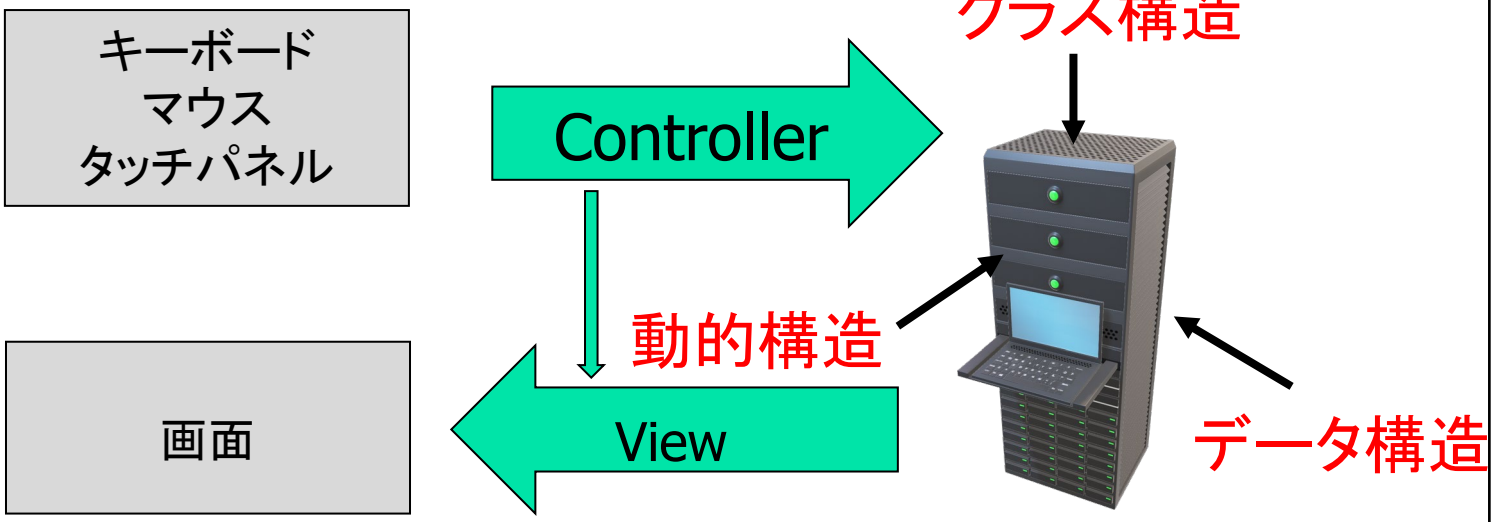
データ構造	ERモデル
	 
		関係と演算
		購買ID | 購買時刻 | カードID | 商品ID | …
		… | … | … | … | …
		3256 | … | 679904 | 788315 | …
		3257 | … | 267910 | 456329 | …
		3258 | … | 779845 | 118365 | …
		… | … | … | … | …
			SQLなど
			`SELECT Region`
			`  ,COUNT(Name)`
			`  ,SUM(Population)`
			`  ,SUM(GNP)`
			`FROM world.country`
			`GROUP BY Region`

クラス構造モデルの例：MVCモデル

- ソフトウェアは入力・処理・出力からなる
- システムを“見える化”“オブジェクト化”にする場合には3つを分割しModel, View, Controllerとしてプログラムにする



三面図とITシステム構造の比較

機械・建築などの三面図	
ソフトウェアの構造	

状態遷移機械モデル

- ITシステムは有限状態遷移機械(有限状態オートマトン, automaton)と仮定
  - 有限種類の内部状態
  - 入力イベントにより状態が遷移すると同時に出力が発生する



[Wikipedia]

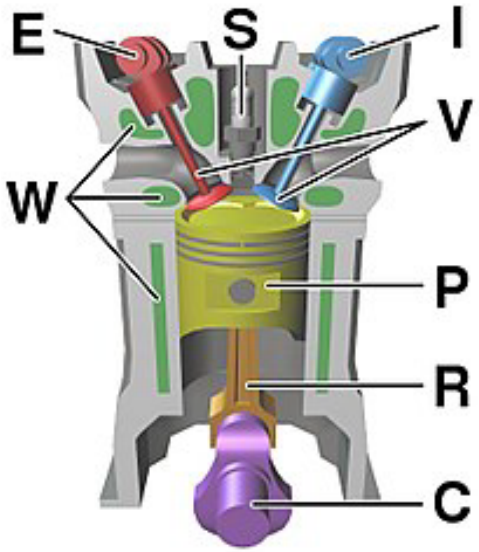


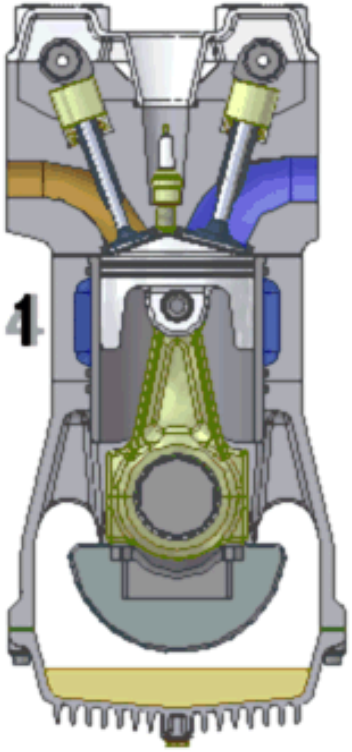
協力:(有)フクオカ機業

中身が見えない機械はコンピュータだけでない

- ガソリンエンジン

  動きを理解するには内部状態の遷移を記述する

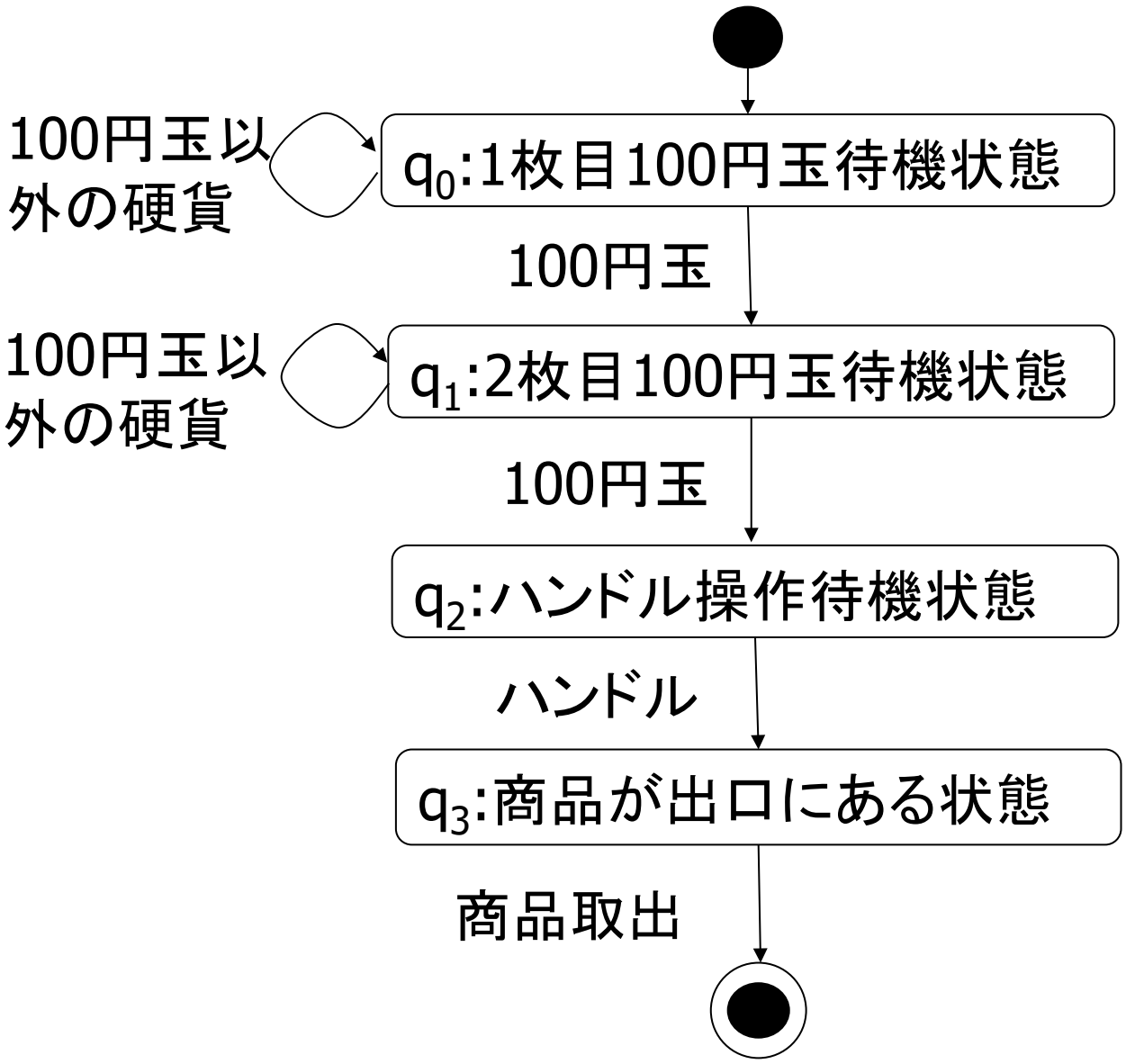




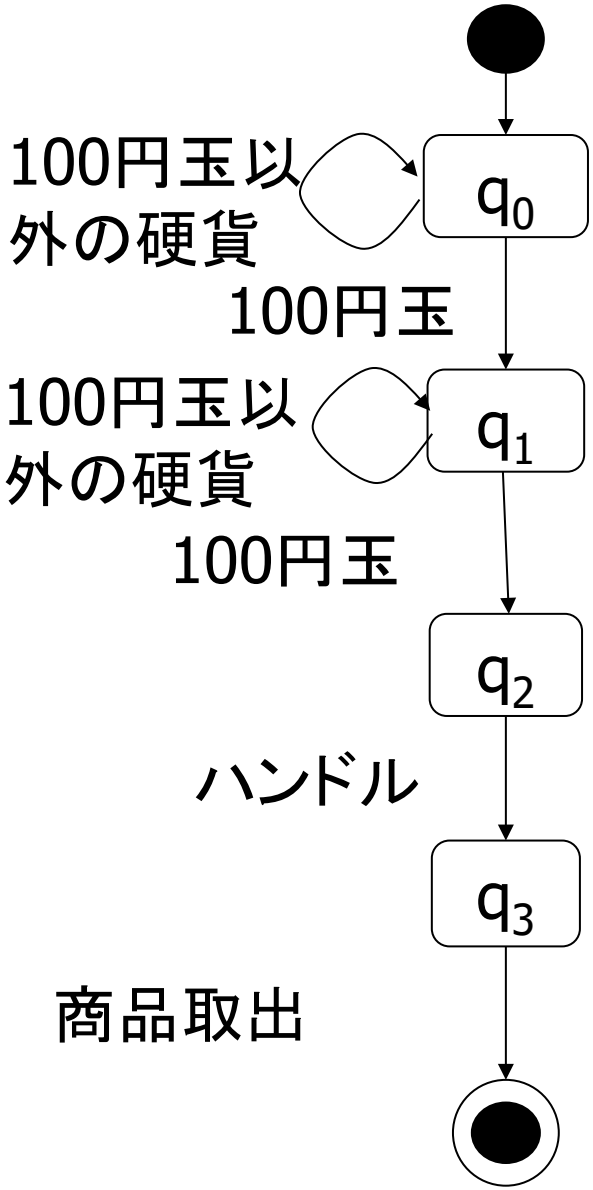
(1)吸入
(2)圧縮
(3)燃焼・膨張
(4)排気

[図はWikipediaより]

100円玉を2枚入れる自動販売機



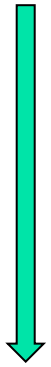
状態遷移表による表現

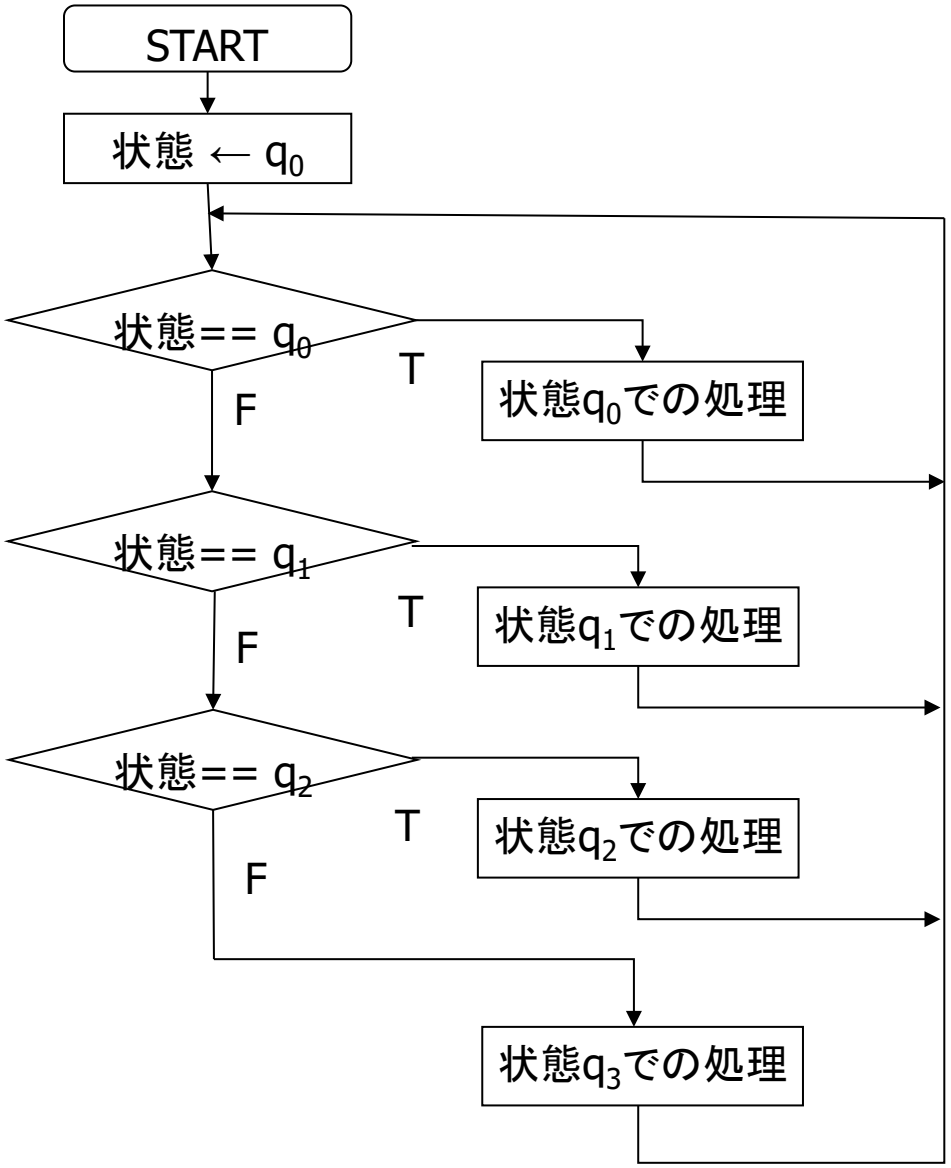


	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_4

状態遷移表からフローチャートへ(1-1)

	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_0



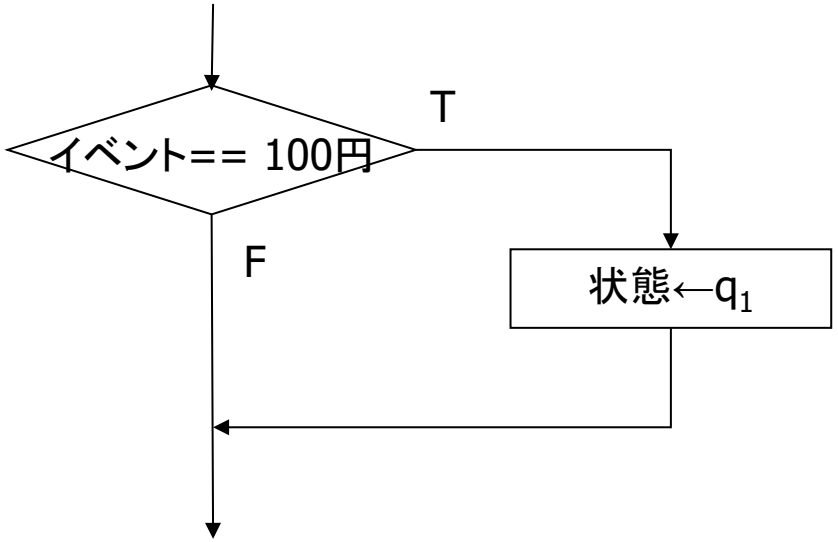


状態を優先して手続きを構成する

状態遷移表からフローチャートへ(1-2)

	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_0

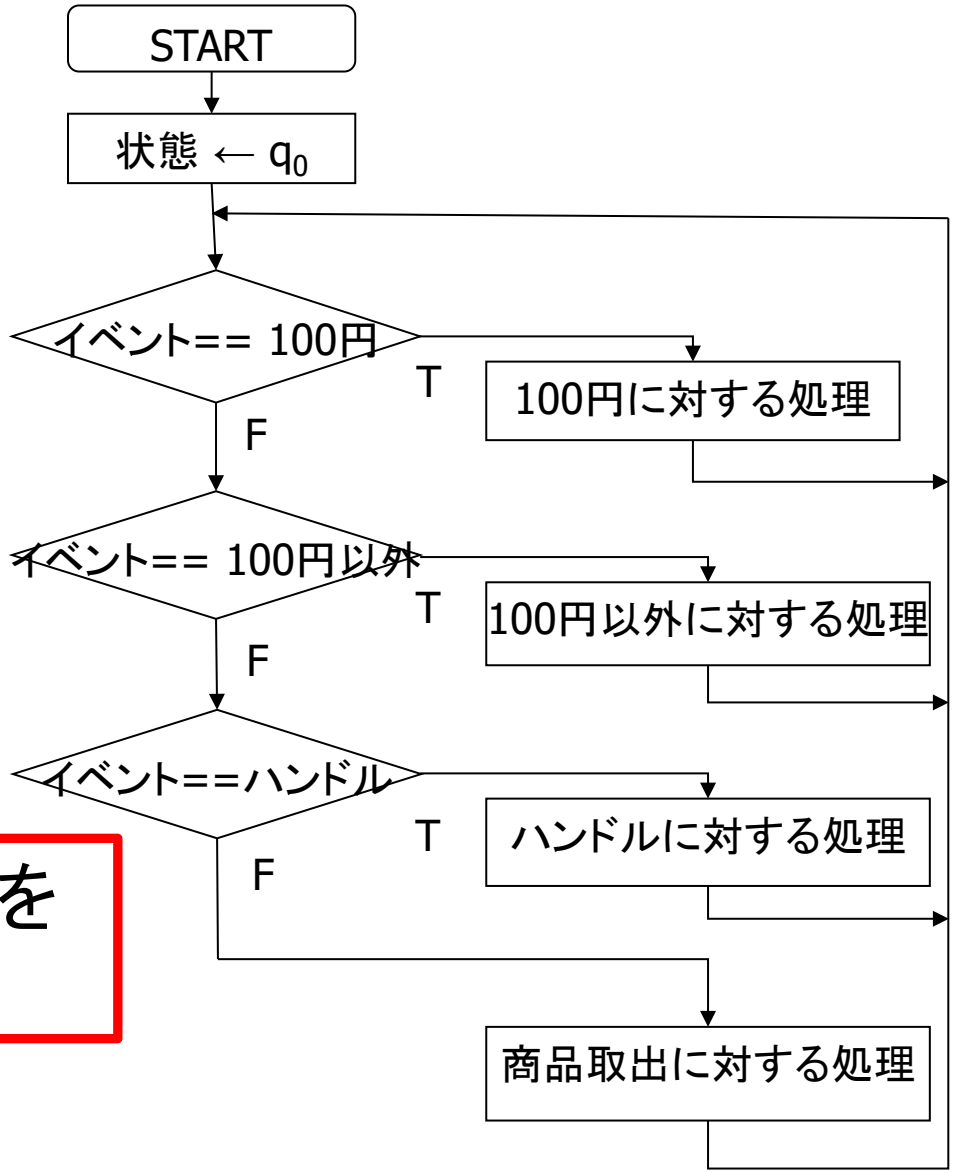
状態q_0での処理の詳細



状態を優先して手続きを構成する

状態遷移表からフローチャートへ(2-1)

	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_0

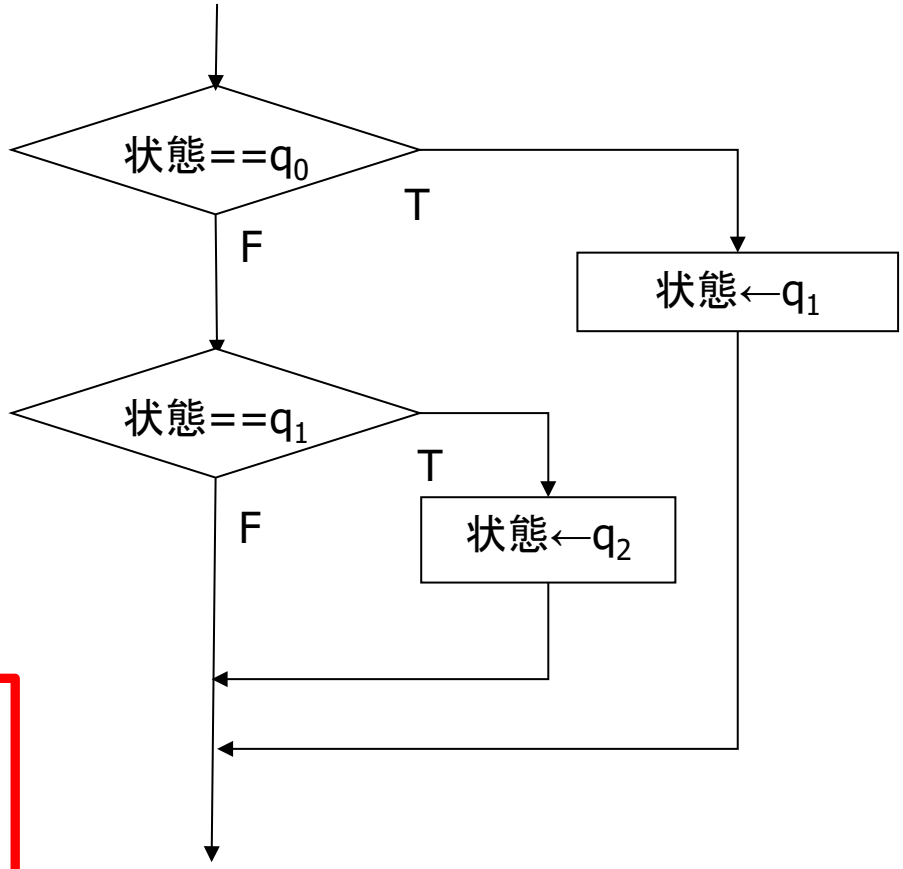


イベントを優先して手続きを構成する

状態遷移表からフローチャートへ(1-2)

	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_0

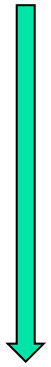
100円玉に対する処理の詳細

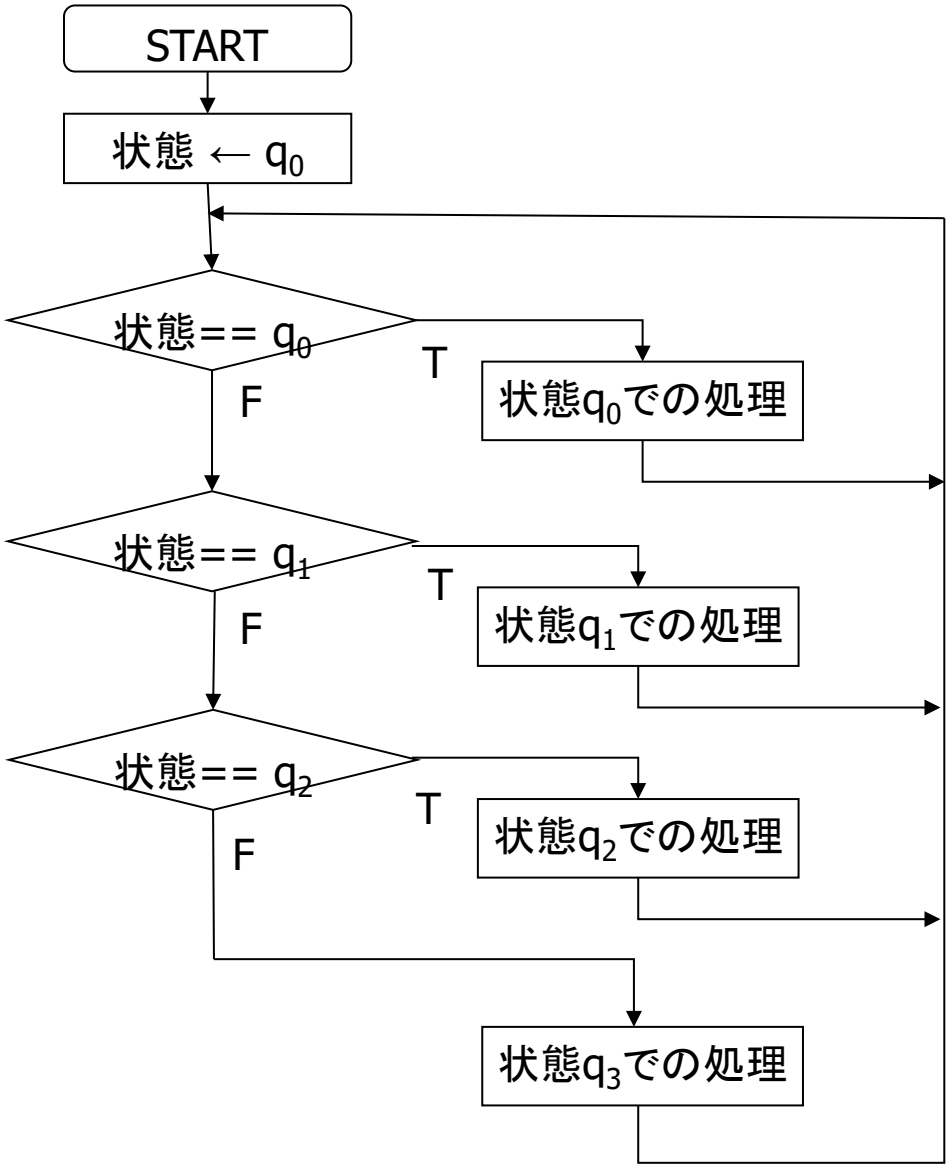


イベントを優先して手続きを構成する

状態遷移表からフローチャートへ(1-1)

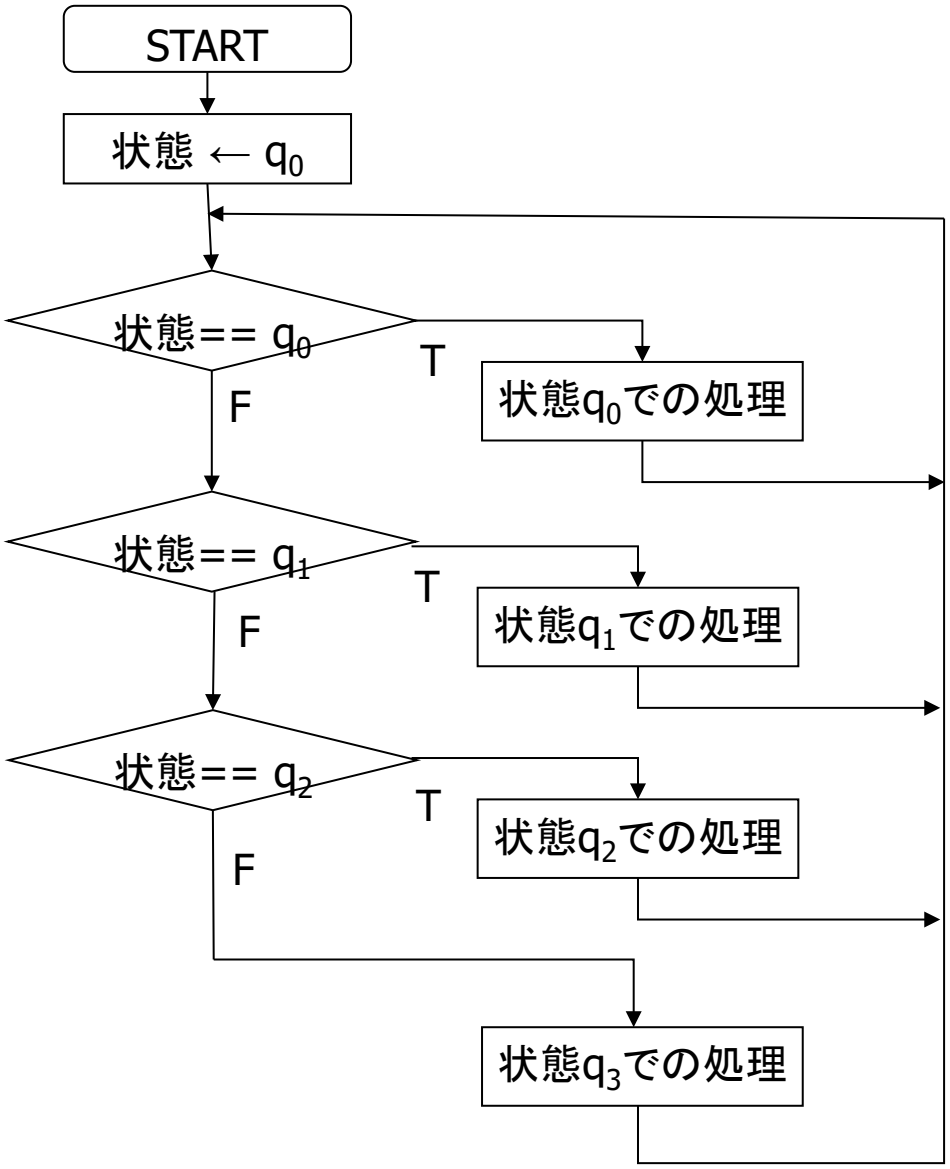
	100	100以外	ハンドル	商品取出
q_0	q_1	q_0		
q_1	q_2	q_1		
q_2			q_3	
q_3				q_0





状態を優先して手続きを構成する

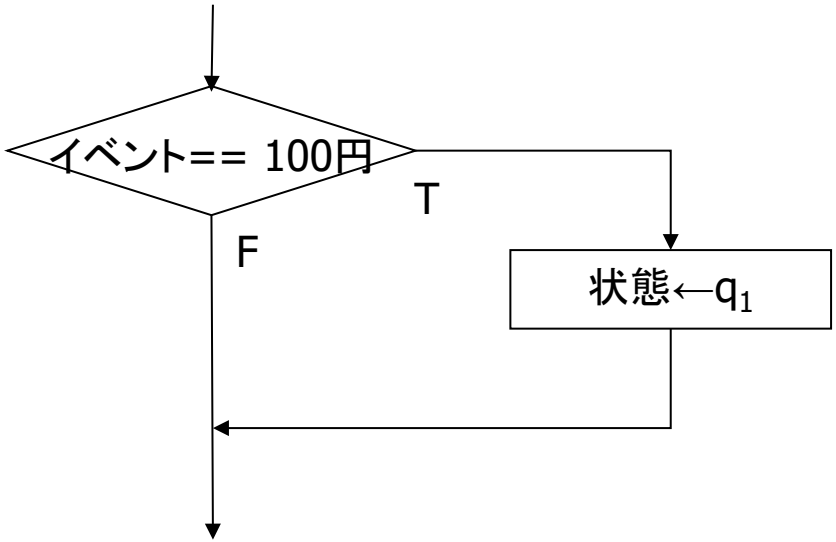
フローチャートからプログラムへ(1-1)




```
q = 0
while (TRUE)
    if ( q == 0 )
            状態q0での処理
    else if ( q == 1 )
            状態q1での処理
    else if ( q == 2)
            状態q2での処理
    else
            状態q2での処理
```

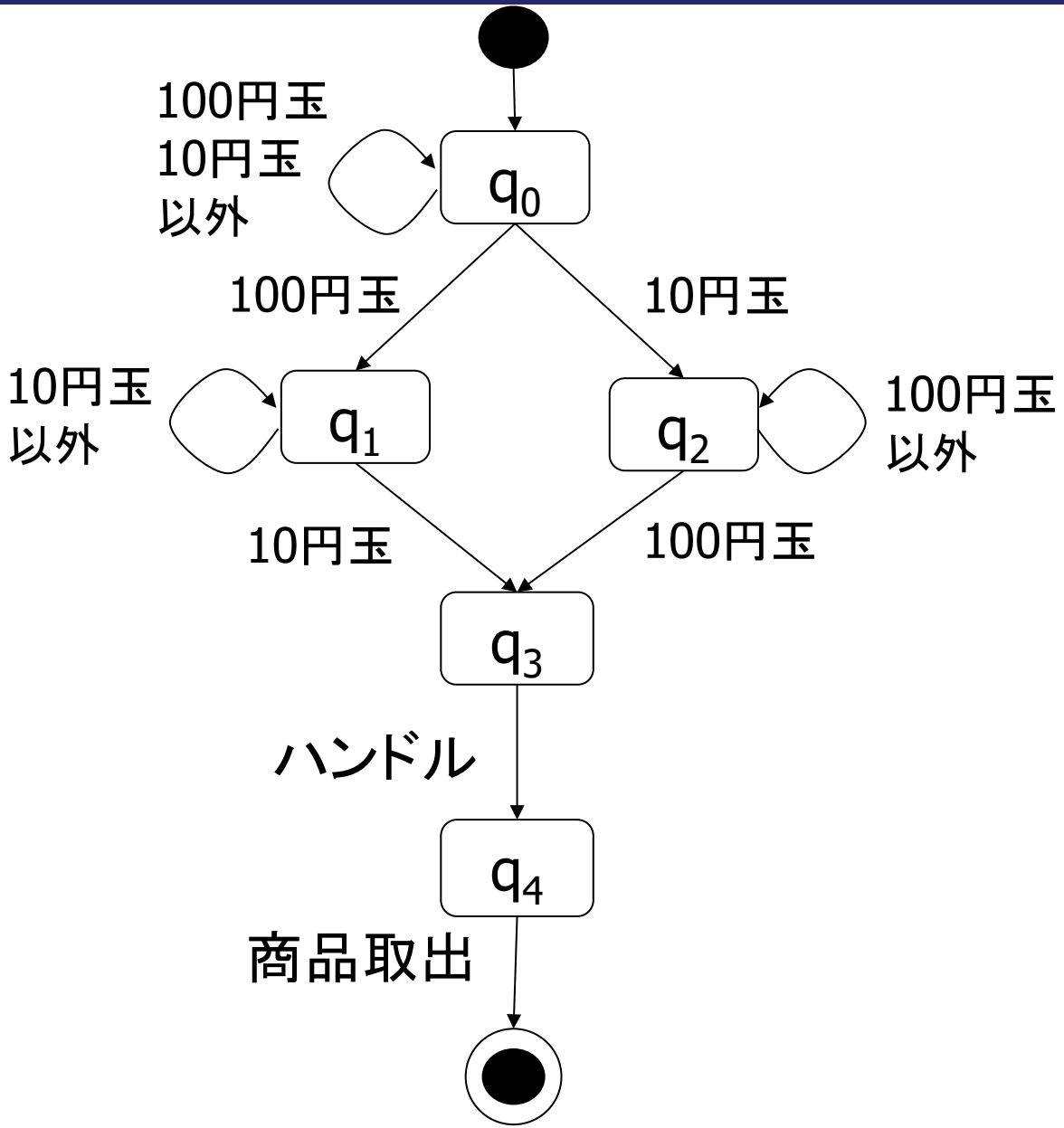
状態遷移表からフローチャートへ(1-2)

状態q_0での処理の詳細

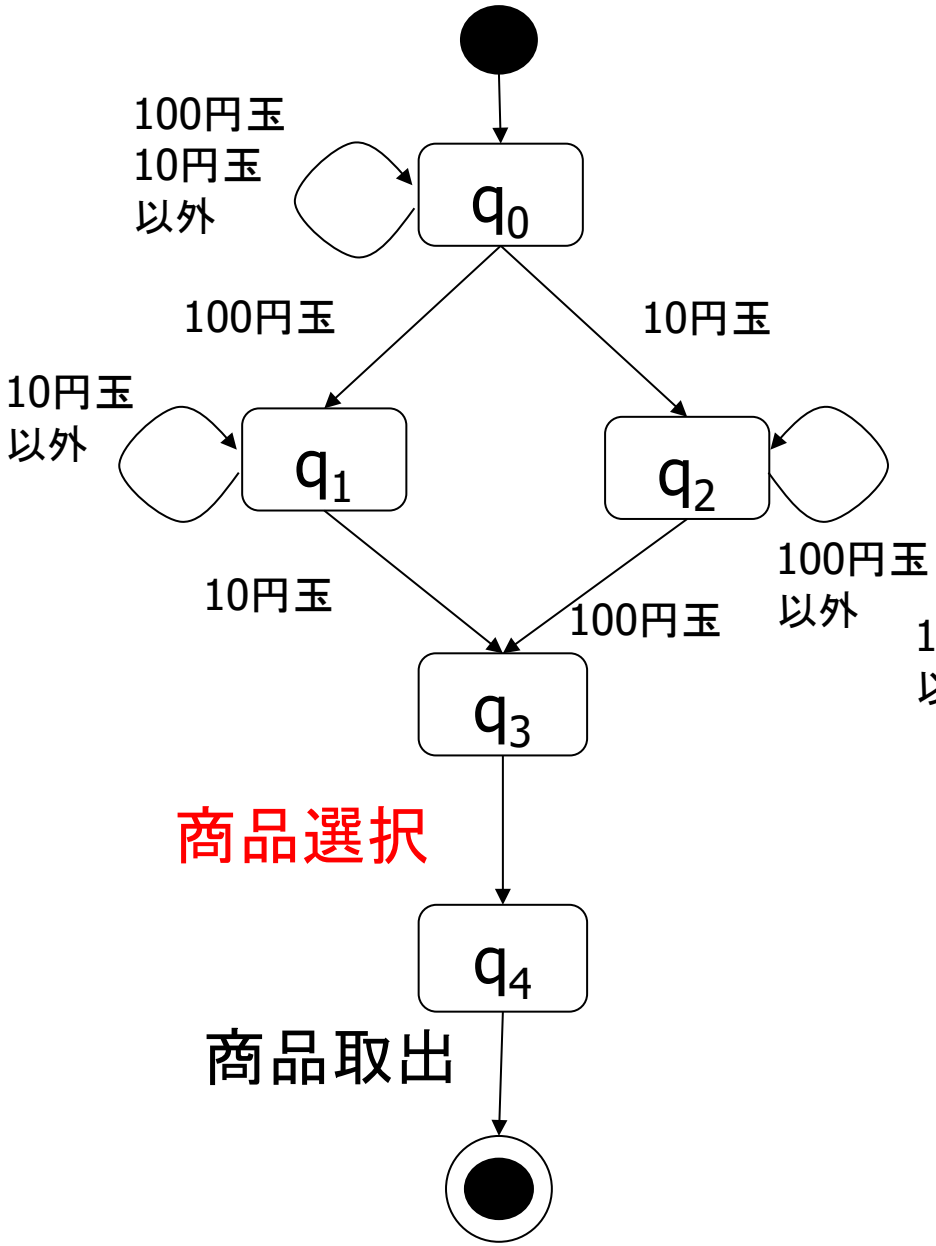


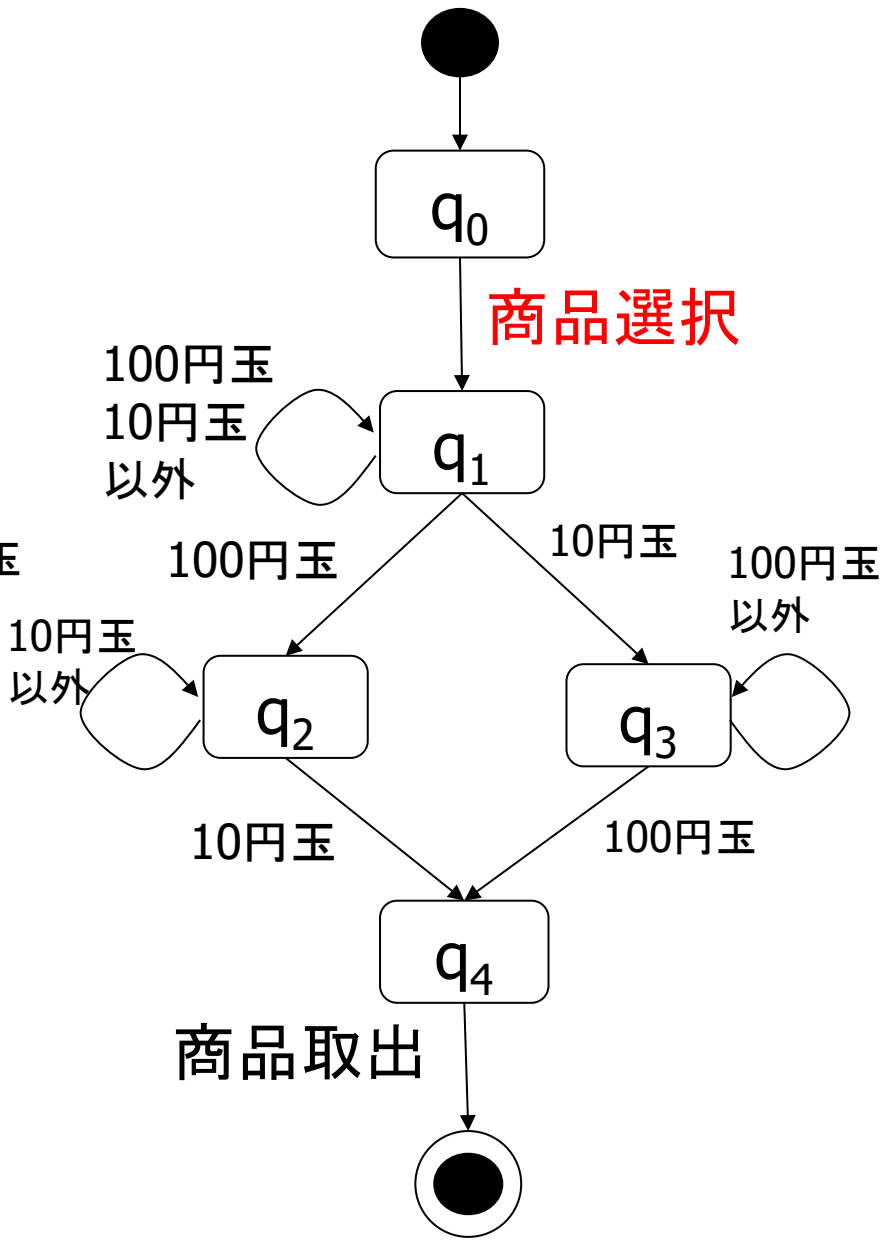

```
e = getevent()
if ( e == OneHundredCoin )
        q = 1
```

100円玉と10円玉各1枚を入れる自動販売機

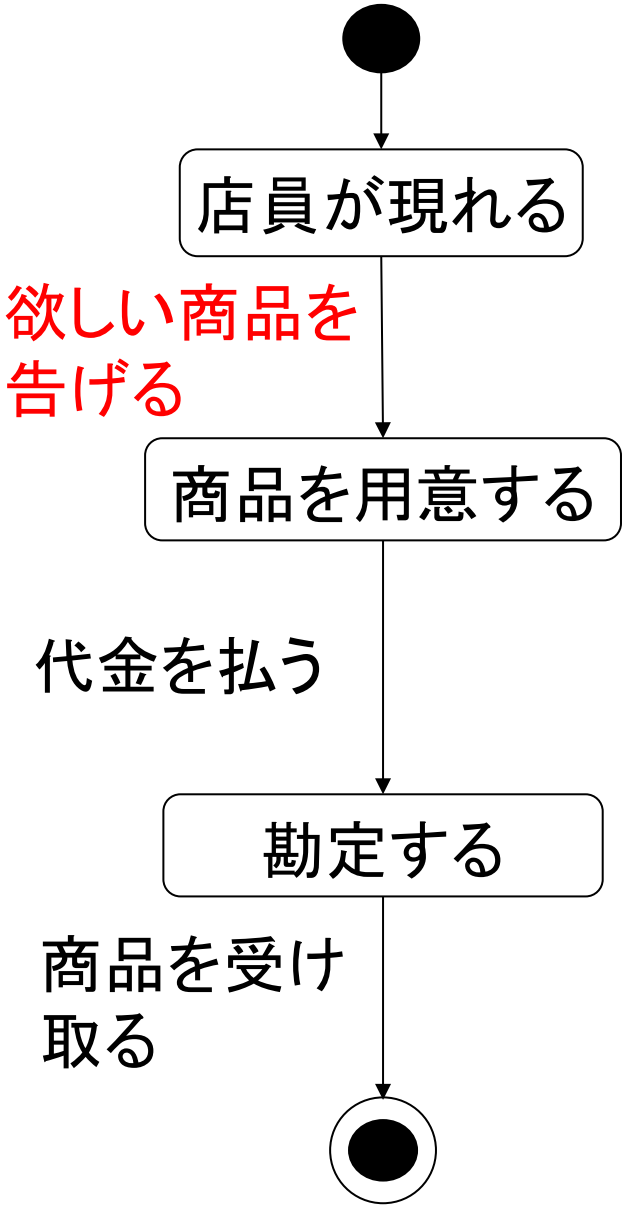


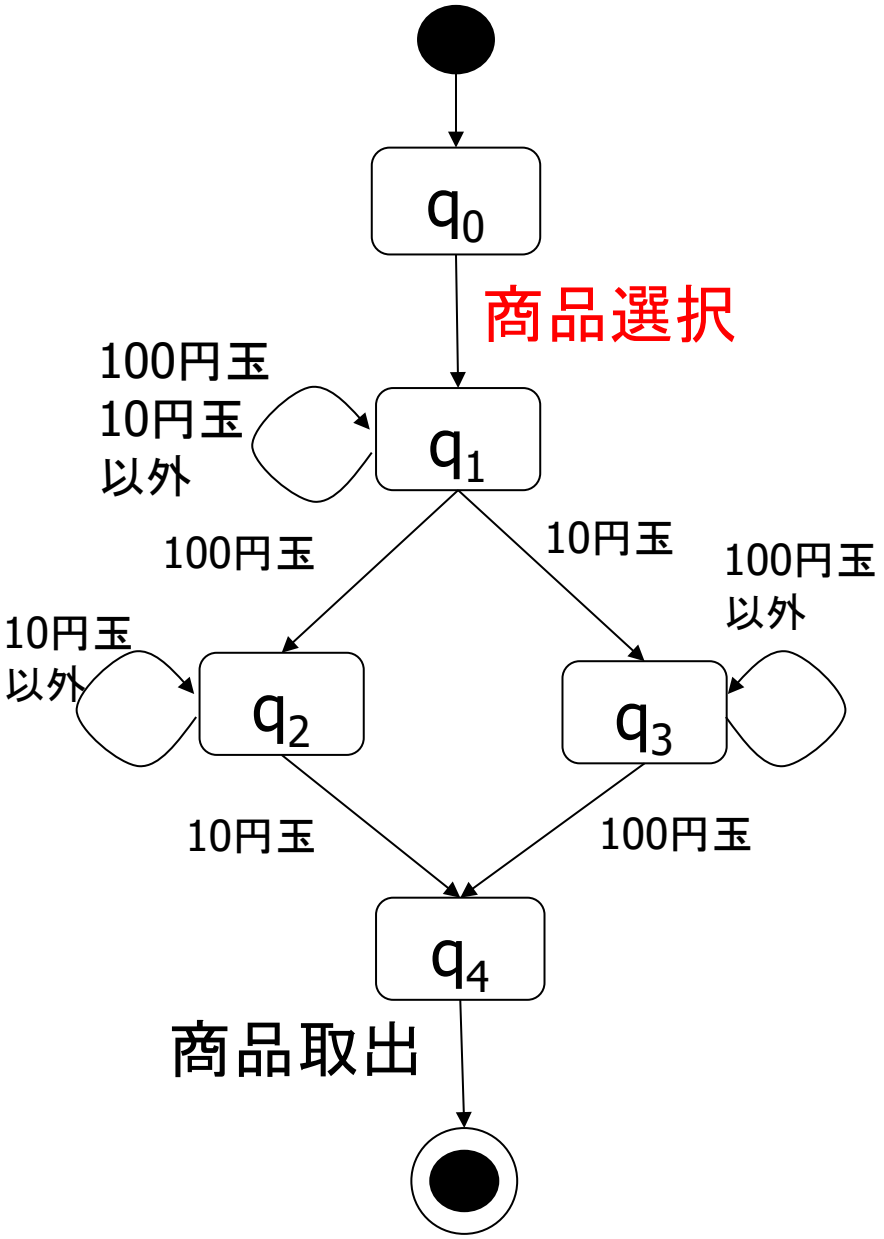
商品選択が先にある自動販売機





店員が商品を売る場合との類比

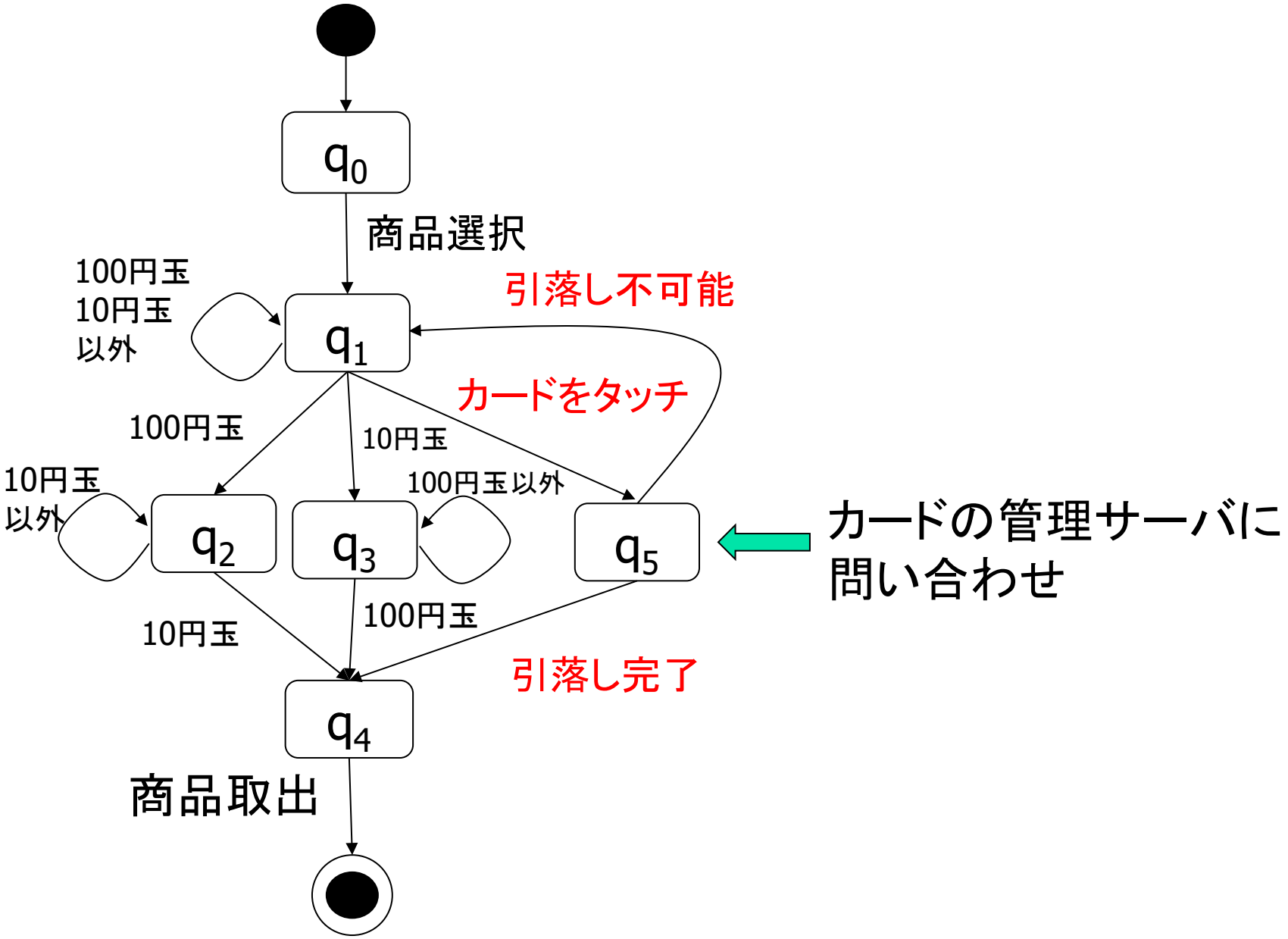




要求仕様の表現手法としての状態遷移機械

- 人間の行為と機械の行為の両方を表すことができる
  - 要求仕様は人間の行為である「業務」の表現
- 段階的に詳細化が可能
- 状態数が増えた場合に，機能を変えることなく簡約化することが可能[Myhill-Nerodeの定理]
  - 二つの状態遷移機械の機能が同じかどうかが判定できる
- フローチャートの意味表現が可能
  - フローチャートを簡約化する場合に，機能を変化させないことを保証する手立てになる

ICカードが使える自動販売機(1)



ICカードが使える自動販売機(1)

